



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий (МОСИТ)

КУРСОВАЯ РАБОТА
по дисциплине «Разработка клиент-серверных мобильных приложений»

Тема курсовой работы: «Разработка клиент-серверного мобильного приложения для обмена сообщениями “Мессенджер”»

Студент группы ИКБО-61-23 Безбородов Никита Михайлович

(подпись)

Руководитель
курсовой работы Преподаватель Степанов П.В.

(подпись)

Работа представлена к защите «06» 06 2026 г.

Допущен к защите «06» 06 2026 г.

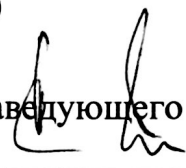
Мещеряков / Мещеряков О.А. /

Москва 2026 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий (МОСИТ)

Утверждаю
И.о. заведующего кафедрой МОСИТ

Головин С.А.
«16» февраля 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы

по дисциплине «Разработка клиент-серверных мобильных приложений»
Студент Безбородов Никита Михайлович
Группа ИКБО-61-23

Тема «Разработка клиент-серверного мобильного приложения для обмена сообщениями “Мессенджер”»

Исходные данные: Разрабатываемый прототип мобильного приложения должен предоставлять возможности полностью интерактивной системы с понятным дружественным UI и обеспечивать необходимую функциональность, которая формируется в зависимости от заданной темы и предметной области изучаемых вопросов.

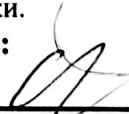
Перечень вопросов, подлежащих разработке, и обязательного графического материала:

1. Описание предметной области мобильного приложения;
2. Сравнительный анализ существующих аналогов мобильного приложения;
3. Обоснование выбора инструментальных средств разработки мобильного приложения;
4. Определение функциональных и нефункциональных требований;
5. Проектирование пользовательского интерфейса мобильного приложения;
6. Логическая модель мобильного приложения (диаграмма в нотации IDEF0, диаграмма классов);
7. Проектирование базы данных мобильного приложения;
8. Разработка клиентской части мобильного приложения;
9. Разработка серверной части мобильного приложения;
10. Тестирование разработанного мобильного приложения.

Отчет по курсовой работе в виде пояснительной записки.

Срок представления к защите курсовой работы:

Задание на курсовую работу выдал


Подпись руководителя

до «30» мая 2026 г.

Степанов П.В.

(ФИО руководителя)

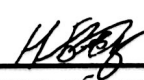
«16» февраля 2026 г.

Безбородов Н.М.

(ФИО обучающегося)

«16» февраля 2026 г.

Задание на курсовую работу получил


Подпись обучающегося

Оглавление

ВВЕДЕНИЕ	4
1. АНАЛИЗ СОСТОЯНИЯ ОБЪЕКТА ИССЛЕДОВАНИЯ	6
1.1. Описание предметной области мобильного приложения.....	6
1.2. Сравнительный анализ существующих аналогов.....	8
1.3. Обоснование выбора инструментальных средств для разработки мобильного приложения.....	11
2.1. Определение функциональных и нефункциональных требований	14
2.1.1. Функциональные требования	14
2.1.2. Нефункциональные требования	15
2.1.3. Use Case диаграмма.....	16
2.2. Проектирование пользовательского интерфейса (макет, реализованный в Figma, описание межэкранного взаимодействия).....	17
2.3. Логическая модель приложения	21
2.3.1 Проектирование мобильного приложения (диаграмма в методологии IDEF0, диаграмма классов).....	21
2.3.2 Проектирование базы данных (логическая модель БД, описание каждой таблицы)	29
ГЛАВА 3. РАЗРАБОТКА МОБИЛЬНОГО ПРИЛОЖЕНИЯ	35
3.1. Разработка клиентской части мобильного приложения	35
3.2. Разработка серверной части мобильного приложения	39
3.2.1 Описание основной логики сервера.....	39
3.2.2 Подключение базы данных к серверу	42
3.2.3 Подключение сервера к клиентской части мобильного приложения	44
3.3. Тестирование разработанного приложения, unit-тестирование, описание полученных результатов)	48
ЗАКЛЮЧЕНИЕ	52
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	54
ПРИЛОЖЕНИЯ.....	55

ВВЕДЕНИЕ

Актуальность темы исследования. В современном цифровом обществе потребность в оперативном обмене информацией, мгновенной коммуникации и поддержании постоянной связи между пользователями постоянно растёт. Традиционные способы общения, такие как: телефонные звонки, SMS-сообщения, электронная почта, зачастую не обеспечивают должной скорости передачи данных, не позволяют организовывать групповое взаимодействие и не поддерживают передачу мультимедийного контента в полном объёме. Существующие мессенджеры либо имеют ограничения по функционалу, либо не обеспечивают должного уровня приватности и безопасности переписки, либо не позволяют гибко комбинировать личные чаты, групповые беседы и каналы для широкого вещания в рамках единого приложения. В связи с этим разработка специализированного клиент-серверного мобильного мессенджера, ориентированного на удобство, скорость, безопасность и расширенный функционал коммуникации, является актуальной задачей.

Объект исследования — процесс организации обмена сообщениями и мультимедийного взаимодействия между пользователями с использованием мобильных устройств.

Предмет исследования — методы и средства разработки клиент-серверного мобильного мессенджера на платформе Android с использованием языка Kotlin и серверных технологий на базе фреймворка Ktor.

Цель курсовой работы — разработать клиент-серверный мобильный мессенджер WhatsMAX, позволяющий пользователям обмениваться текстовыми сообщениями и файлами, совершать аудио- и видеозвонки, создавать групповые чаты и каналы, а также получать push-уведомления в реальном времени.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и существующих аналогов мобильных мессенджеров (Telegram, WhatsApp, Viber);

- обосновать выбор инструментальных средств для разработки клиентской и серверной частей приложения;
- сформулировать функциональные и нефункциональные требования к разрабатываемому приложению;
- спроектировать пользовательский интерфейс и логическую модель приложения, включая структуру базы данных;
- реализовать клиентскую часть мобильного приложения на языке Kotlin с использованием Jetpack Compose и Clean Architecture;
- реализовать серверную часть приложения на Ktor и обеспечить её взаимодействие с клиентом через REST API и WebSocket, а также с базой данных PostgreSQL;
- выполнить тестирование разработанного приложения (не менее 10 тестовых сценариев);
- оформить пояснительную записку в соответствии с требованиями ГОСТ и подготовить материалы для защиты.

Структура курсовой работы. Работа состоит из введения, трёх глав, заключения, списка использованной литературы и приложений.

В первой главе проводится анализ предметной области, рассматриваются существующие аналоги мессенджеров, обосновывается выбор технологий разработки.

Во второй главе определяются функциональные и нефункциональные требования, проектируется пользовательский интерфейс, разрабатывается логическая модель приложения.

В третьей главе описывается процесс разработки клиентской части на Kotlin и серверной части на Ktor, подключение базы данных PostgreSQL к серверу, интеграция Firebase Cloud Messaging и WebRTC, а также результаты тестирования.

В заключении подводятся итоги работы, формулируются выводы о достижении поставленной цели и предлагаются возможные направления дальнейшего развития приложения.

1. АНАЛИЗ СОСТОЯНИЯ ОБЪЕКТА ИССЛЕДОВАНИЯ

1.1. Описание предметной области мобильного приложения

Предметная область разработки охватывает процессы организации обмена сообщениями, мультимедийного взаимодействия и аудио-/видеосвязи между пользователями с использованием мобильных устройств. Под мессенджером в данном контексте понимается программная система, обеспечивающая мгновенную передачу текстовых сообщений, файлов и медиаданных, организацию групповых бесед и каналов вещания, а также установление аудио- и видеосоединений между участниками в режиме реального времени.

Мобильные мессенджеры находят применение в самых разных сценариях: в корпоративной среде для оперативной коммуникации сотрудников и обмена рабочими файлами, в образовательных учреждениях для организации учебных групп и каналов с объявлениями, в личной жизни для поддержания связи с родственниками и друзьями, а также в общественных и развлекательных проектах для информирования широкой аудитории через каналы. Несмотря на существование множества популярных мессенджеров (Telegram, WhatsApp, Viber и другие), большинство из них обладают рядом ограничений.

Эти решения либо имеют закрытую серверную архитектуру, не позволяющую развернуть систему на собственной инфраструктуре, либо привязаны к конкретному номеру телефона и облачной экосистеме провайдера, либо не предоставляют в рамках одного приложения полный набор функций: личные чаты, группы, каналы вещания, а также аудио- и видеозвонки.

Кроме того, многие сервисы не позволяют пользователю контролировать хранение данных и не дают разработчикам возможности расширять функционал под собственные нужды.

Разрабатываемое в рамках курсовой работы приложение WhatsMAX призвано устранить перечисленные недостатки. Оно представляет собой нативное клиент-серверное решение для операционной системы Android, в котором пользователи проходят обязательную регистрацию и аутентификацию

через Firebase Auth, обмен сообщениями защищён авторизацией по JWT-токенам, а доставка сообщений и обновление онлайн-статусов осуществляется в реальном времени через WebSocket-соединение.

Серверная часть реализована самостоятельно на базе фреймворка Ktor и может быть развёрнута на произвольной инфраструктуре, а клиентская часть на Jetpack Compose поддерживает работу с локальной базой данных Room для офлайн-режима, push-уведомления через Firebase Cloud Messaging и установление аудио-/видеозвонков по технологии WebRTC.

Основными сущностями предметной области являются: пользователь (физическое лицо с учётной записью и профилем), чат (совокупность участников и обмена сообщениями между ними, который может быть личным или групповым), канал (односторонний поток сообщений от автора к подписчикам по аналогии с Telegram), сообщение (единица обмена, содержащая текст, файл или медиа, с поддержкой операций reply/edit/delete) и звонок (сеанс аудио- или видеосвязи между двумя или более участниками). Взаимодействие между ними организуется через серверное REST API и WebSocket-канал, которые принимают запросы от мобильного клиента, проверяют их корректность, сохраняют данные в реляционной базе PostgreSQL и рассылают обновления всем подключённым устройствам. Такая архитектура обеспечивает единый источник достоверных данных, синхронизацию переписки между устройствами одного пользователя и исключает расхождения информации.

Таким образом, предметная область мобильных мессенджеров является практически востребованной, а разработка специализированного нативного приложения с собственной серверной частью позволяет закрыть ряд недостатков существующих универсальных решений и обеспечить полный контроль над данными и инфраструктурой системы.

1.2. Сравнительный анализ существующих аналогов

Сравнительный анализ служит для всесторонней оценки существующих решений с точки зрения их функциональных возможностей, технической архитектуры, пользовательского опыта и экономических факторов. Он позволяет: выявить сильные и слабые стороны каждого продукта, определить ключевые требования, которым должно соответствовать разрабатываемое приложение, построить обоснованную аргументацию выбора архитектуры и набора технологий на основе объективных данных.

В рамках данного исследования были отобраны четыре популярных мессенджера: Telegram, WhatsApp, Viber и Signal. Также проведено их сравнение по критериям, отражающим основные потребности пользователей: наличие нативного мобильного приложения, поддержка групповых чатов и каналов, аудио- и видеосвязь, открытость архитектуры, привязка к номеру телефона и условия использования (см. в таблице 1.1).

Telegram — облачный мессенджер с открытым клиентским кодом и широкой экосистемой ботов и каналов. Сервис обеспечивает мгновенную доставку сообщений, поддерживает групповые чаты до 200 000 участников, каналы для односторонней рассылки, аудио- и видеозвонки, а также секретные чаты с E2E-шифрованием. Платформа имеет нативные приложения для Android и iOS, поддерживает редактирование и удаление сообщений. Основными недостатками являются закрытая серверная архитектура (исходный код сервера не опубликован, развернуть собственный сервер невозможно) и обязательная привязка учётной записи к номеру мобильного телефона.

WhatsApp — наиболее распространённый в мире мессенджер, принадлежащий компании Meta. Сервис обеспечивает сквозное шифрование всех сообщений и звонков, поддерживает групповые чаты до 1024 участников, передачу файлов с ограничением до 2 ГБ, аудио- и видеозвонки. Основные недостатки: отсутствие полноценных каналов вещания (заменены статусами и сообществами с ограничениями), закрытая архитектура без возможности самостоятельного развёртывания, обязательная привязка к номеру теле-

фона и SIM-карте, ограниченные возможности редактирования сообщений после отправки.

Viber — мессенджер с акцентом на голосовую связь и звонки между обычными телефонными номерами. Платформа предоставляет нативные мобильные приложения, поддерживает сообщества (аналог каналов), стикеры и игры. Существенным недостатком является наличие рекламных вставок в бесплатной версии и закрытая архитектура. Авторизация привязана к номеру телефона. Viber ценен прежде всего для коммуникации с пользователями за рубежом через VoIP, но не подходит для задач, требующих полного контроля над данными и инфраструктурой.

Signal — мессенджер с акцентом на приватность и безопасность переписки. Платформа использует собственный криптографический протокол Signal Protocol и имеет частично открытую архитектуру: протокол и клиентский код опубликованы, однако официальный сервер централизован и развернуть полностью независимую инфраструктуру в общем случае затруднительно. Signal не поддерживает каналы вещания и публичные сообщества, ориентирован на закрытое общение в небольших группах. Авторизация также привязана к номеру телефона. Сервис бесплатен и существует за счёт пожертвований.

Результаты сравнения сведены в таблицу 1.1.

Таблица 1.1 – Сравнительный анализ существующих аналогов

Критерий	Telegram	WhatsApp	Viber	Signal	Наш проект
Нативное Android-приложение	Да	Да	Да	Да	Да
Личные и групповые чаты	Да	Да	Да	Да	Да
Каналы (broadcast)	Да	Нет	Сообщества	Нет	Да
Аудио- и видеозвонки	Да	Да	Да	Да	Да
Reply/Edit/Delete сообщений	Да	Частично	Частично	Частично	Да

Продолжение Таблицы 1.1

Критерий	Telegram	WhatsApp	Viber	Signal	Наш проект
Открытая архитектура (self-hosted сервер)	Нет	Нет	Нет	Частично	Да, свой сервер на Ktor
Привязка к номеру телефона	Да	Да	Да	Да	Нет (email через Firebase Auth)
Условия использования	Бесплатно	Бесплатно	Бесплатно, с рекламой	Бесплатно (донаты)	Бесплатно

Все рассмотренные мессенджеры предоставляют удобные нативные приложения для быстрого обмена сообщениями и звонками, однако ни один из них не позволяет конечному пользователю или организации развернуть полноценную систему на собственной инфраструктуре. Ключевым недостатком аналогов является обязательная привязка учётной записи к номеру мобильного телефона, что делает невозможным использование мессенджера без SIM-карты и ставит приватность пользователя в зависимость от оператора связи. Кроме того, все коммерческие сервисы имеют закрытую серверную архитектуру, не позволяющую интегрировать мессенджер с корпоративными системами или гибко расширять его функционал.

Разрабатываемый проект «Мессенджер WhatsMAX» выделяется открытой архитектурой с собственным сервером на Ktor и REST API, отсутствием обязательной привязки к номеру телефона (авторизация через Firebase Auth по email), полным набором функций в едином приложении (личные чаты, группы, каналы, аудио- и видеозвонки на WebRTC, push-уведомления через FCM) и поддержкой операций reply/edit/delete над сообщениями. Это позволит пользователям более гибко управлять перепиской, разворачивать систему на собственной инфраструктуре и расширять функционал по мере необходимости.

1.3. Обоснование выбора инструментальных средств для разработки мобильного приложения

Для реализации клиент-серверного мобильного мессенджера WhatsMAX был выбран следующий стек технологий, соответствующий требованиям задания и современным тенденциям разработки.

Язык программирования Kotlin выбран для разработки как серверной, так и клиентской части. Kotlin является современным статически типизированным языком, полностью совместимым с Java, но предоставляющим более лаконичный синтаксис, безопасную работу с nullable-типами, поддержку корутин для асинхронного программирования и другие современные возможности. Кроме того, использование единого языка для обеих частей системы упрощает разработку, отладку и поддержку кода, а также позволяет переиспользовать модели данных между клиентом и сервером.

Клиентская часть реализована на платформе Android с использованием Jetpack Compose — современного декларативного фреймворка для построения пользовательского интерфейса. Этот выбор обусловлен преимуществами Compose: сокращение количества кода по сравнению с XML-разметкой, удобная работа с состоянием, встроенная поддержка темной темы и лучшая производительность при перерисовке списков чатов и сообщений. Архитектура клиентского приложения построена в соответствии с принципами Clean Architecture, предусматривающей разделение на слои data, domain и presentation. Такой подход обеспечивает независимость бизнес-логики от фреймворков, упрощает тестирование и поддержку кода. Для внедрения зависимостей используется библиотека Hilt, а кодогенерация выполняется через KSP (Kotlin Symbol Processing) вместо устаревшего kapt, что обеспечивает совместимость с Gradle 8.x и значительно ускоряет сборку проекта.

Серверная часть разработана с использованием фреймворка Ktor 2.3.7, который представляет собой легковесный асинхронный фреймворк для построения микросервисов и API на Kotlin. Его преимуществами являются:

встроенная поддержка корутин, минималистичность, гибкость конфигурации, удобная работа с сериализацией JSON, а также нативная поддержка

WebSocket-соединений «из коробки». В качестве веб-сервера используется встроенный Netty, что позволяет обрабатывать большое количество одновременных подключений и обеспечивает низкую задержку доставки сообщений. Сервер запускается на порту 8080.

База данных — PostgreSQL выбрана как промышленная реляционная СУБД с открытым исходным кодом, поддерживающая ACID-транзакции, продвинутые типы данных (JSONB, массивы), полнотекстовый поиск и горизонтальное масштабирование.

PostgreSQL обеспечивает надёжное хранение учётных записей пользователей, переписки, информации о чатах, каналах и звонках. Локальная база whatsapp развёрнута под пользователем postgres. Взаимодействие с БД организовано через ORM Exposed — лёгкую и типобезопасную библиотеку от JetBrains, которая позволяет писать запросы на чистом Kotlin без потери производительности и без необходимости работы с громоздкими аннотациями JPA.

Реальное время и доставка сообщений. Для мгновенной доставки сообщений, обновления онлайн-статусов и индикаторов «печатает» используется протокол WebSocket: на сервере — стандартный модуль Ktor WebSockets, на клиенте — OkHttp WebSocket. REST API на основе Retrofit применяется для операций, не требующих постоянного соединения (авторизация, загрузка истории, управление профилем). Для аудио- и видеозвонков интегрирована технология WebRTC, при этом сервер выполняет роль сигнального узла для обмена SDP-офферами и ICE-кандидатами между участниками.

Аутентификация и push-уведомления. Регистрация и вход пользователей реализованы через Firebase Authentication, что обеспечивает безопасное хранение паролей, поддержку восстановления доступа и интеграцию с другими сервисами Google.

Для доставки push-уведомлений о новых сообщениях и входящих звонках используется Firebase Cloud Messaging (FCM): на сервере подключён Firebase Admin SDK, на клиенте — клиентская часть FCM. Для работы Firebase в проект помещены файлы `backend/firebase-service-account.json` и `android/app/google-services.json`.

Локальное хранилище и офлайн-режим. На стороне клиента используется библиотека Room — обёртка над SQLite от Google, обеспечивающая локальное кэширование чатов, сообщений и контактов. Это позволяет пользователю просматривать переписку и работать с приложением даже при отсутствии интернет-соединения, а при восстановлении связи Room синхронизируется с сервером через WebSocket.

Библиотеки и инструменты. Для сериализации данных используется `kotlinx.serialization`, для управления состоянием экранов — `ViewModel` и `StateFlow`, для асинхронных операций — корутины Kotlin. Система сборки — Gradle с зафиксированной версией `wrapper 8.4`. Пакет Android-приложения — `com.whatsmax`, `applicationId` — `com.whatsmax.messenger`. Для подключения эмулятора к локальному серверу в `BASE_URL` и `WS_URL` используется специальный адрес `10.0.2.2`, проксирующий запросы на хост-машину разработчика.

Выбранный стек технологий полностью соответствует заданию на курсовую работу, современным стандартам разработки и позволяет реализовать все требуемые функциональные возможности: регистрацию и авторизацию через Firebase Auth, обмен текстовыми сообщениями и файлами в реальном времени через WebSocket, групповые чаты и каналы вещания, аудио- и видеозвонки на WebRTC, push-уведомления через FCM, операции `reply/edit/delete` над сообщениями, а также обеспечить высокое качество кода благодаря Clean Architecture и Hilt DI.

2. ПРОЕКТИРОВАНИЕ МОБИЛЬНОГО ПРИЛОЖЕНИЯ

2.1. Определение функциональных и нефункциональных требований (описание функциональных требований, use case диаграмма, описание нефункциональных требований)

2.1.1. Функциональные требования

Функциональные требования описывают, какие действия должно уметь выполнять приложение.

Таблица 2.1.1 – Функциональные требования

Идентификатор	Требование
Авторизация и управление аккаунтом	
F1	Регистрация нового пользователя по email и паролю через Firebase Auth
F2	Вход в существующий аккаунт
F3	Выход из аккаунта
F4	Пароль при регистрации должен содержать минимум 6 символов
F5	Редактирование профиля пользователя (имя, аватар, статус)
Управление чатами	
F6	Просмотр списка всех чатов текущего пользователя
F7	Создание личного чата с другим пользователем
F8	Создание группового чата (название, список участников)
F9	Просмотр информации о чате и списка его участников
F10	Выход из группового чата
Обмен сообщениями	
F11	Отправка текстовых сообщений в чат
F12	Отправка файлов и медиа (фото, видео, документы)
F13	Получение сообщений в реальном времени через WebSocket
F14	Ответ на сообщение (reply) с цитированием исходного
F15	Редактирование собственного отправленного сообщения
F16	Удаление собственного сообщения
F17	Отображение онлайн/офлайн статуса собеседника
F18	Индикатор «печатает...» при наборе сообщения собеседником
Каналы и вещание	
F19	Создание канала вещания с названием и описанием
F20	Подписка на каналы и просмотр их списка
F21	Публикация сообщений в канале (только владельцем канала) истории поиска
Звонки	
F22	Совершение аудиозвонка 1-на-1 на технологии WebRTC
F23	Совершение видеозвонка 1-на-1 на технологии WebRTC
F24	Приём входящих звонков с экраном уведомления
Уведомления и интерфейс	
F25	Push-уведомления о новых сообщениях через Firebase Cloud Messaging
F26	Push-уведомления о входящих звонках
F27	Поиск чатов и контактов по имени

Продолжение Таблицы 2.1.1

F28	Переключение между светлой и тёмной темой оформления
F29	Отображение индикатора загрузки при выполнении сетевых операций
F30	Плейсхолдер «Нет сообщений» при пустом чате

2.1.2. Нефункциональные требования

Нефункциональные требования описывают, каким должно быть приложение с точки зрения качества, производительности и надёжности.

Таблица 2.1.2 – Нефункциональные требования

Категория	Идентификатор	Требование
Производительность	NF1	Задержка доставки сообщения через WebSocket не должна превышать 200 мс при стабильном соединении
	NF2	Время ответа REST API при загрузке списка чатов не должно превышать 500 мс в среднем
	NF3	Приложение должно запускаться на устройстве не более чем за 3 секунды
Надёжность	NF4	Время безотказной работы серверной части не менее 24 часов при тестовой нагрузке
	NF5	При потере сети пользователю показывается понятное сообщение об ошибке, локальная переписка остаётся доступна из Room
	NF6	Автоматическое переподключение WebSocket при восстановлении сетевого соединения
Безопасность	NF7	Аутентификация пользователей реализуется через Firebase Authentication
	NF8	Токен авторизации передаётся в заголовке Authorization: Bearer <token> и проверяется на сервере через Firebase Admin SDK
	NF9	Файлы Firebase service account и google-services.json не публикуются в репозитории и подключаются вручную
	NF10	CORS на сервере настроен на ограниченный список origin'ов
Совместимость	NF11	Сервер собирается в Docker-образ на основе eclipse-temurin:17-jre-jammy (multi-stage build)
	NF12	Клиент работает на Android версии 8.0 (API 26) и выше, целевой API 34
	NF13	Эмулятор подключается к локальному серверу через адрес 10.0.2.2:8080
Масштабируемость	NF14	База данных PostgreSQL может быть вынесена на отдельный хост через переменные окружения
	NF15	Стек сервиса разворачивается через docker-compose (backend + PostgreSQL)

Продолжение Таблицы 2.1.2

Удобство использования	NF16	Весь интерфейс выполнен на русском языке
	NF17	Тема оформления приложения автоматически следует системным настройкам устройства (светлая/тёмная)
Поддерживаемость	NF18	Код Android-приложения организован по принципам Clean Architecture (data/domain/presentation)
	NF19	Внедрение зависимостей реализовано через Hilt с кодогенерацией KSP
	NF20	Бизнес-логика клиента вынесена в Use Case-классы домена
	NF21	Gradle wrapper зафиксирован на версии 8.4 для воспроизводимости сборки
Эффективность	NF22	Локальный кэш чатов и сообщений хранится в Room, обеспечивая офлайн-доступ к переписке
	NF23	Приложение не удерживает в памяти лишние данные после закрытия экрана (использование viewModelScope и StateFlow)

2.1.3. Use Case диаграмма

Диаграмма вариантов использования показывает, какие действия может выполнять пользователь в системе.

Актор: пользователь — зарегистрированное лицо, прошедшее авторизацию через Firebase Authentication.

Варианты использования:

- зарегистрироваться — создание нового аккаунта по email и паролю через Firebase Auth;
- войти в систему — авторизация существующего пользователя;
- выйти из системы — завершение сессии и удаление локального токена;
- редактировать профиль — изменение имени, аватара и статуса пользователя;
- просмотреть список чатов — получение списка всех личных и групповых чатов текущего пользователя;
- создать личный чат — открытие диалога 1-на-1 с другим пользователем;

- создать групповой чат — создание группы с произвольным названием и списком участников;
- отправить сообщение — передача текстового сообщения собеседнику или в группу через WebSocket;
- отправить файл/медиа — передача фото, видео или документа;
- редактировать или удалить сообщение — изменение или удаление ранее отправленного собственного сообщения (reply/edit/delete);
- создать канал — создание канала вещания, в котором пользователь становится владельцем;
- подписаться на канал — добавление канала в список подписок и получение публикаций автора;
- совершить аудио- или видеозвонок — установление сеанса связи с собеседником по технологии WebRTC через сигнальный сервер;
- скать чаты и контакты — фильтрация списка чатов и поиск пользователей по имени.

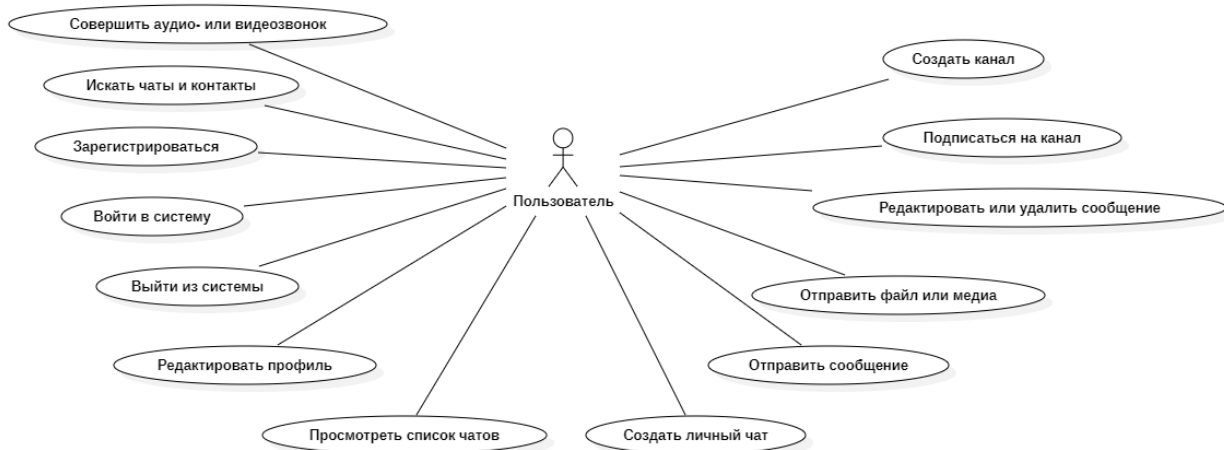


Рисунок 2.1.1 – Use Case диаграмма

2.2. Проектирование пользовательского интерфейса (макет, реализованный в Figma, описание межэкранного взаимодействия)

Проектирование пользовательского интерфейса и пользовательского опыта выполнялось с использованием инструмента Figma.

На основе разработанных макетов в Figma была реализована навигация между экранами в самом приложении, а также определён внешний вид всех ключевых элементов интерфейса. Финальные варианты экранов закреплены в исходниках в виде Compose Preview-функций (файл ScreenPreviews.kt), что позволяет верстать и согласовывать макеты непосредственно в Android Studio без необходимости запуска приложения на устройстве.

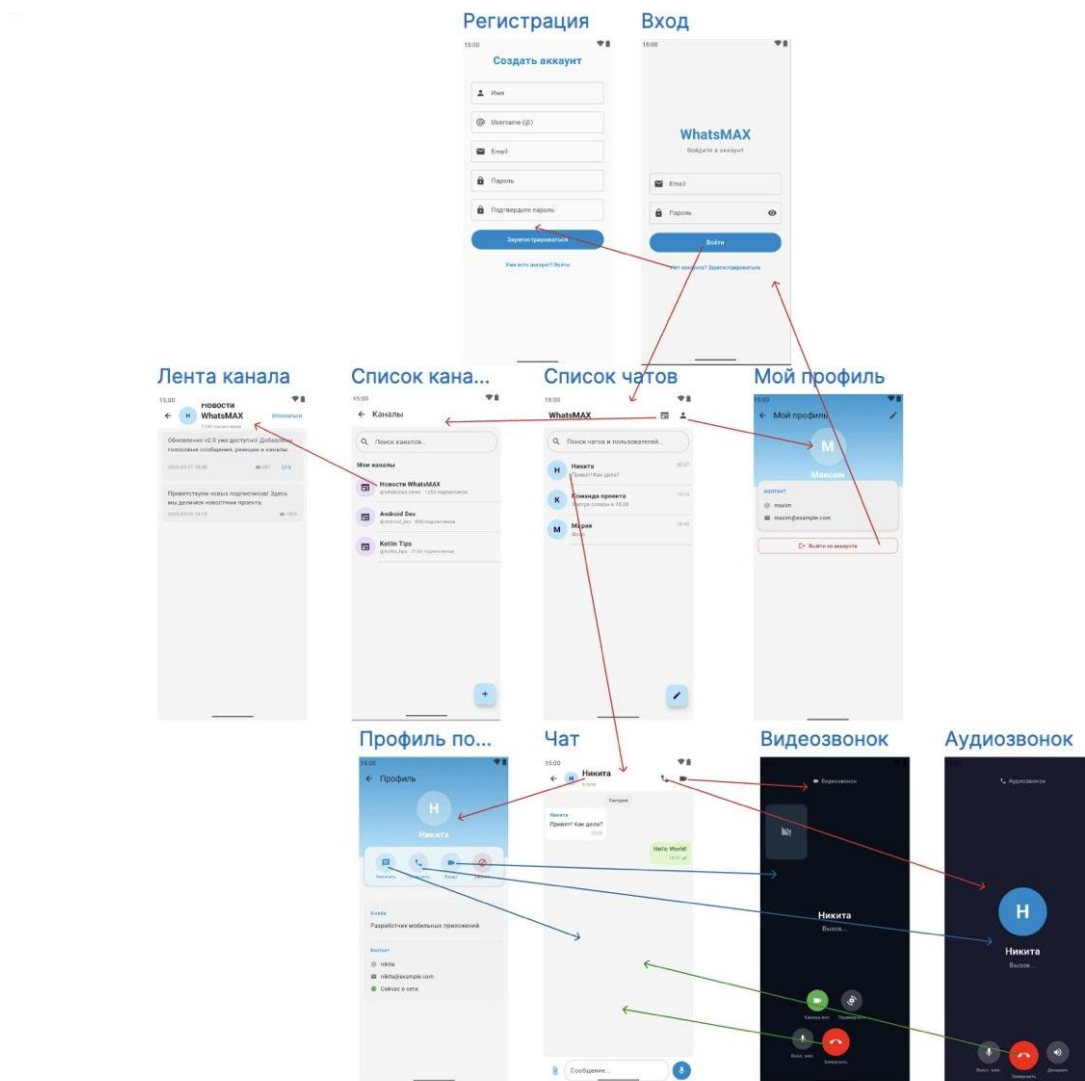


Рисунок 2.2 – Макет приложение, реализованный в figma

На рисунке 2.2 представлены десять ключевых экранов мессенджера WhatsMAX, описывающих схему межэкранного взаимодействия. Согласно этой схеме, при первом запуске приложения пользователь попадает на экран входа, выполненный в минималистичном стиле: логотип «WhatsMAX» круп-

ным шрифтом фирменного синего цвета, подзаголовок «Войдите в аккаунт», два поля ввода (Email с иконкой конверта, Пароль с иконкой замка и кнопкой переключения видимости), основная кнопка «Войти» во всю ширину экрана и текстовая ссылка «Нет аккаунта? Зарегистрироваться». При повторных запусках приложения, если пользователь не выходил из аккаунта, авторизационный токен Firebase Auth сохраняется в локальном хранилище, и пользователь автоматически перенаправляется на главный экран.

Экран регистрации содержит пять полей ввода с соответствующими иконками: Имя, Username (@), Email, Пароль и Подтверждение пароля. Все поля занимают полную ширину экрана и расположены в прокручиваемой колонке. Внизу размещены основная кнопка «Зарегистрироваться» и текстовая ссылка «Уже есть аккаунт? Войти». После успешной регистрации через Firebase Authentication пользователь автоматически перенаправляется на главный экран — список чатов.

Главный экран (список чатов) содержит верхнюю панель `AppBar` с заголовком «WhatsMAX» и двумя кнопками-иконками справа — «Каналы» и «Профиль». В правом нижнем углу расположена плавающая кнопка `FloatingActionButton` с иконкой карандаша для создания нового чата. Основную часть экрана занимает закруглённая поисковая строка с плейсхолдером «Поиск чатов и пользователей...», а под ней — список чатов в виде строк. Каждая строка содержит круглый аватар (52 dp) с первой буквой имени собеседника, имя жирным шрифтом, время последнего сообщения, и снизу — превью последнего сообщения серым цветом. Строки разделяются тонкими горизонтальными линиями с отступом слева под аватар.

Экран чата имеет составной заголовок: кнопка возврата, круглый аватар собеседника (36 dp), его имя и статус «в сети» зелёным цветом, а также две кнопки действий справа — иконки телефона и видеокамеры для запуска аудио- и видеозвонка по WebRTC. В центре экрана отображается лента сообщений: системные метки даты («Сегодня») в виде закруглённых плашек посередине, чужие сообщения слева с именем отправителя и временем, свои

сообщения справа в зелёном пузырьке с двойной галочкой подтверждения доставки. Внизу — панель ввода с иконкой прикрепления файла, текстовым полем «Сообщение...» с закруглёнными углами и круглой кнопкой микрофона/отправки фирменного синего цвета.

Экран аудиозвонка оформлен в тёмной цветовой схеме с тёмно-синим фоном. Сверху по центру — иконка телефона и подпись «Аудиозвонок», в центре — крупный круглый аватар (120 dp) фирменного синего цвета с буквой собеседника, под ним имя крупным шрифтом и подпись статуса «Вызов...». Внизу располагается ряд из трёх круглых кнопок управления: переключение микрофона, крупная красная кнопка завершения звонка и переключение динамика.

Экран видеозвонка использует более тёмный сине-чёрный фон, имитирующий видеопоток собеседника. В левом верхнем углу расположено маленькое окно (108×144 dp) с собственной камерой пользователя. Сверху — иконка видеокамеры и подпись «Видеозвонок», в центре — имя собеседника и статус «Вызов...». В нижней части экрана размещены два ряда кнопок: верхний — переключение камеры и её перевертывание, нижний — микрофон, завершение звонка (крупная красная кнопка) и пустое место для симметрии.

Экран собственного профиля оформлен с градиентной шапкой в фирменных синих тонах. В центре шапки — круглый аватар с первой буквой имени на полупрозрачном фоне, имя пользователя крупным жирным шрифтом, username с символом @ и индикатор онлайн-статуса в виде зелёной точки с подписью «в сети». Под шапкой расположена карточка «КОНТАКТ» с информационными строками (username, email), а ниже — окантованная кнопка «Выйти из аккаунта» красного цвета с иконкой Logout. В правом верхнем углу шапки — кнопка редактирования профиля.

Экран профиля другого пользователя построен по схожей структуре, но вместо кнопки выхода содержит панель быстрых действий, поднятую над контентом в виде белой карточки с тенью: «Написать», «Позвонить», «Видео» и «Заблок.» (красным цветом). Ниже расположены две тонированные

карточки: «О себе» с описанием и «Контакт» с username, email и текущим статусом «Сейчас в сети» зелёным цветом.

Экран списка каналов содержит заголовок «Каналы» в TopAppBar с кнопкой возврата, плавающую кнопку добавления нового канала, поисковую строку с плейсхолдером «Поиск каналов...» и подзаголовок «Мои каналы». Список каналов состоит из строк с квадратными иконками-аватарами на вторичном цвете с пиктограммой газеты, именем канала жирным шрифтом и подписью формата «@handle · N подписчиков» серым цветом.

Экран ленты канала имеет составной заголовок с аватаром канала, его названием и числом подписчиков, а справа — текстовая кнопка «Отписаться». Публикации канала отображаются в виде карточек Material3: основной текст публикации, под ним — дата и время в нижнем левом углу, а в правом — количество просмотров (иконка глаза) и количество комментариев (иконка пузыря). При нажатии на счётчик комментариев открывается экран обсуждения публикации.

2.3. Логическая модель приложения

2.3.1 Проектирование мобильного приложения (диаграмма в методологии IDEF0, диаграмма классов)

Методология IDEF0 применяется для моделирования функциональной структуры системы. Она позволяет визуализировать ключевые процессы, определить их входы, выходы, механизмы и управляющие воздействия. С помощью диаграмм IDEF0 можно наглядно отразить, какие функции выполняет мессенджер WhatsMAX, как они взаимодействуют между собой и с внешними объектами.

Контекстная диаграмма описывает приложение как единый процесс, взаимодействующий с внешними сущностями. В качестве входов выступают «Email и пароль», «Текст сообщения», «Прикреплённый файл», «Поисковый запрос» и «Сигнальные данные звонка (SDP/ICE)». В качестве выходов — «Результат регистрации/входа», «Список чатов и каналов», «Доставленное сообщение», «Событие реального времени через WebSocket» и «Сеанс аудио-

/видеозвонка». Механизм работы задают пользователь, мобильное Android-приложение, серверная часть на Ktor, база данных PostgreSQL, Firebase Authentication и стек WebRTC. Управление обеспечивается набором правил: правилами обмена сообщениями, правилами работы с паролями, правилами работы с каналами, а также Федеральным законом «О персональных данных» (152-ФЗ).

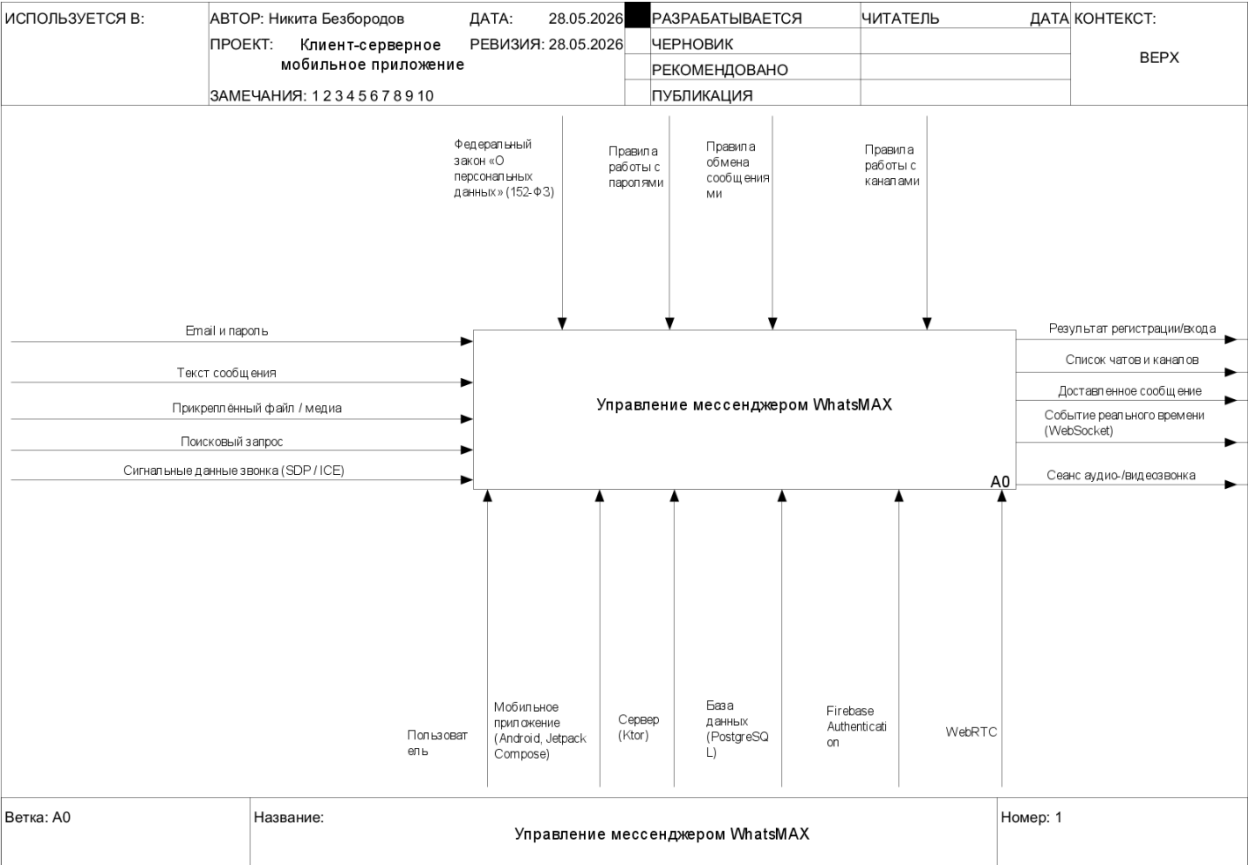


Рисунок 2.3.1 – Контекстный уровень диаграммы IDEF0

Декомпозируем контекст на четыре ключевых подпроцесса: «Регистрация и авторизация пользователя», «Управление чатами и каналами», «Обмен сообщениями» и «Аудио- и видеозвонки».

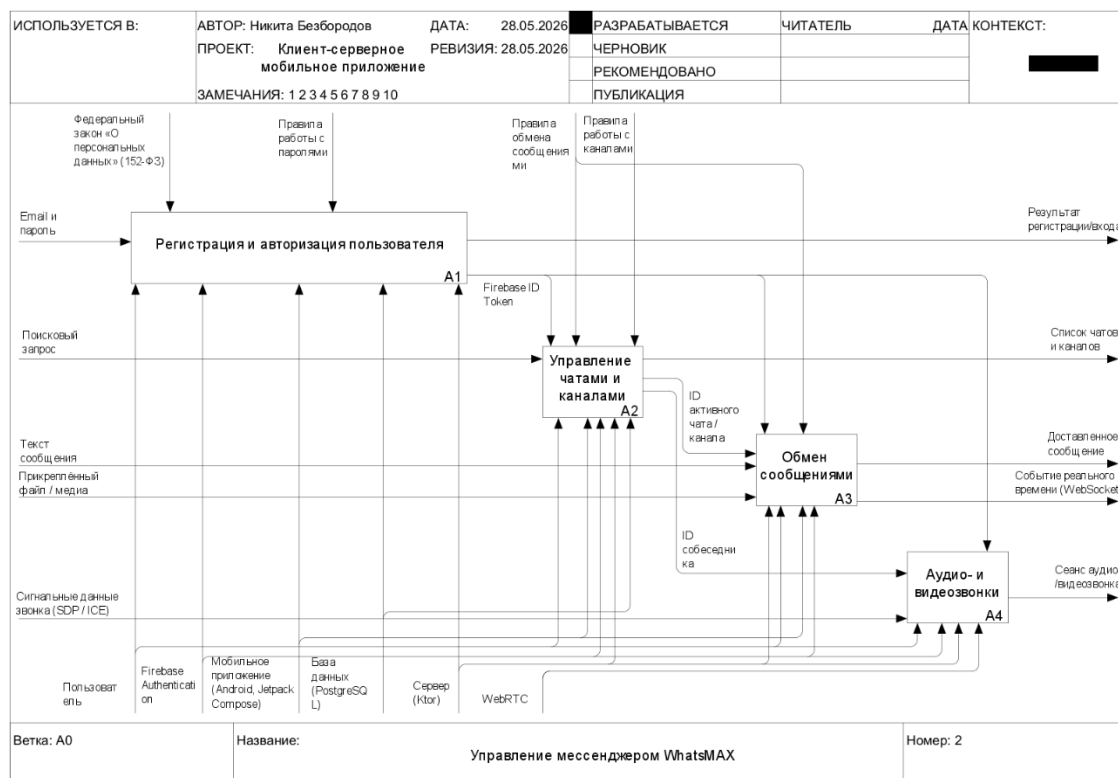


Рисунок 2.3.2 – Декомпозиция контекстного уровня

На уровне A1 представлена декомпозиция блока «Регистрация и авторизация пользователя». В него входят три подпроцесса: «Валидация введенных данных», который принимает email и пароль и проверяет их на соответствие формату и длине; «Регистрация нового пользователя», обеспечивающая создание учетной записи в Firebase Authentication и сохранение профиля в PostgreSQL через сервер Ktor; «Авторизация существующего пользователя», отвечающая за проверку учетных данных и выдачу Firebase ID-токена, который в дальнейшем передается в заголовке Authorization: Bearer <token> при каждом запросе к API.

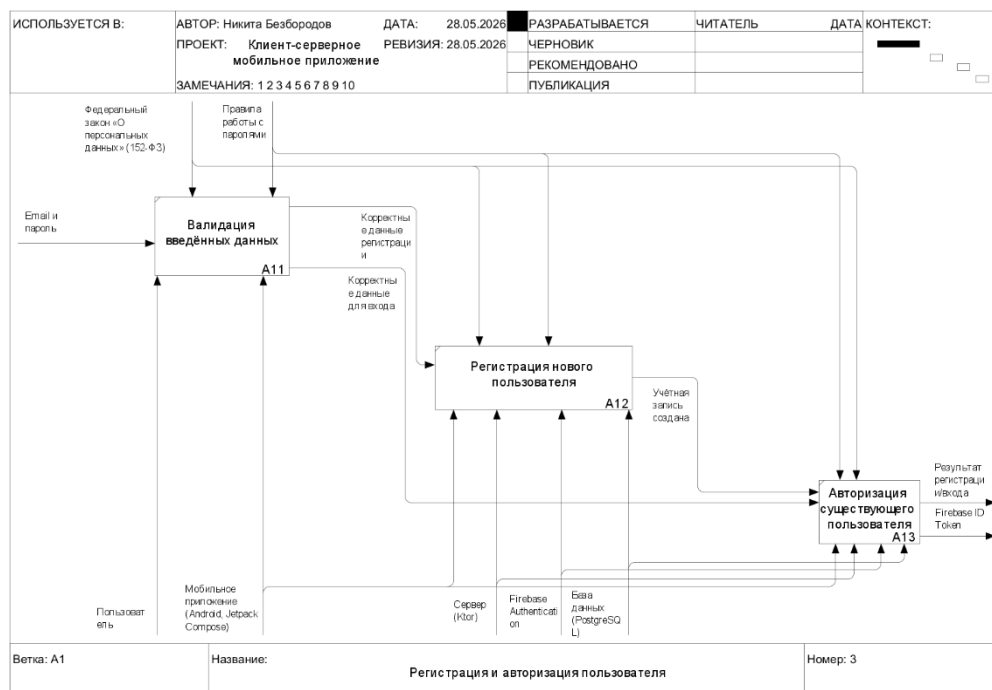


Рисунок 2.3.3 – Декомпозиция процесса «Регистрация и авторизация пользователя»

На уровне A2 представлена декомпозиция блока «Управление чатами и каналами». Он включает четыре подпроцесса: «Создание чата или канала» — добавление нового личного чата, группы или канала с проверкой уникальности handle для каналов; «Просмотр списка чатов и каналов» — отображение всех чатов пользователя с превью последнего сообщения и его подписок на каналы; «Редактирование чата или канала» — изменение названия, описания, состава участников; «Удаление чата / выход из группы / отписка от канала». При каждом из действий Firebase ID-токен пользователя используется для идентификации и проверки прав (только владелец канала может его удалить, только участник чата может из него выйти).

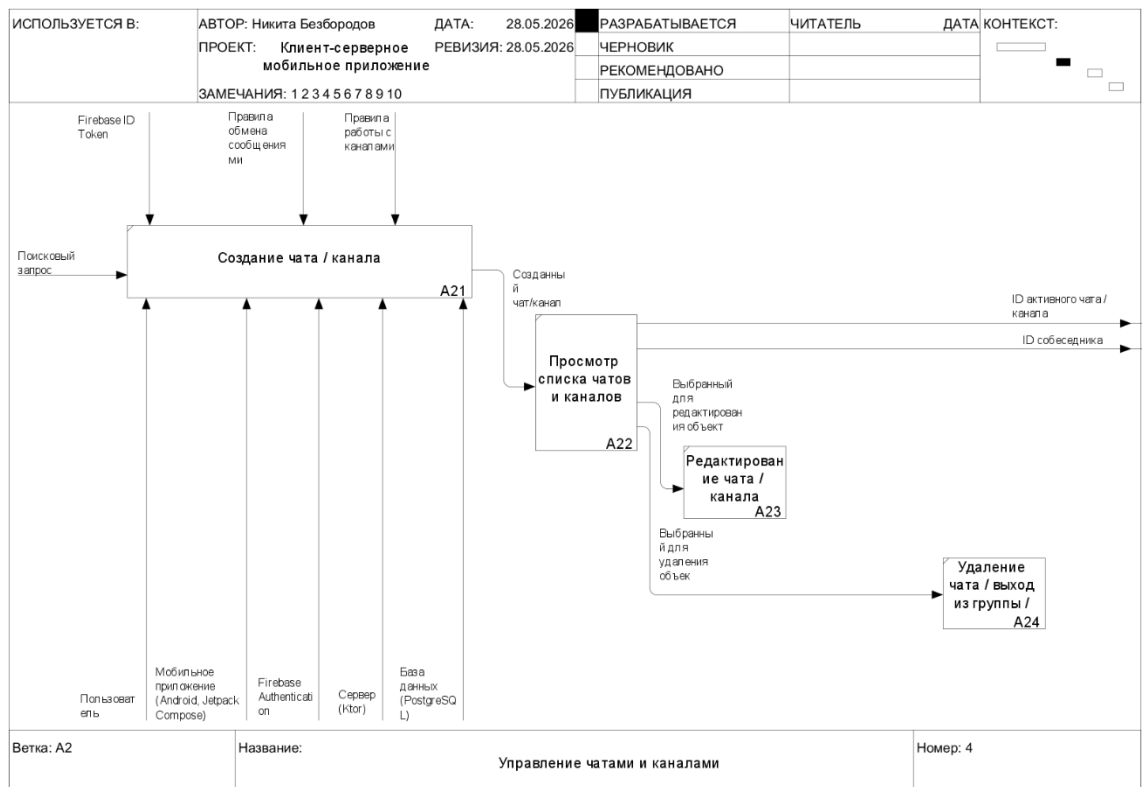


Рисунок 2.3.4 – Декомпозиция процесса «Управление чатами и каналами»

На уровне A3 представлена декомпозиция блока «Обмен сообщениями». Этот процесс разбит на четыре подпроцесса: «Подготовка сообщения» — сборка текста, медиа и опциональной ссылки на цитируемое сообщение replyToId; «Загрузка файла на сервер» — отправка медиа на эндпоинт /files с получением fileId; «Отправка сообщения через WebSocket / REST» — сохранение в PostgreSQL и рассылка событий всем подключённым устройствам участников чата в реальном времени; «Редактирование или удаление собственного сообщения» — изменение поля content с установкой флага isEdited, либо логическое удаление через isDeleted. На всех шагах используется Firebase ID-токен, а управление осуществляется правилами обмена сообщениями (например, редактировать и удалять может только автор сообщения).

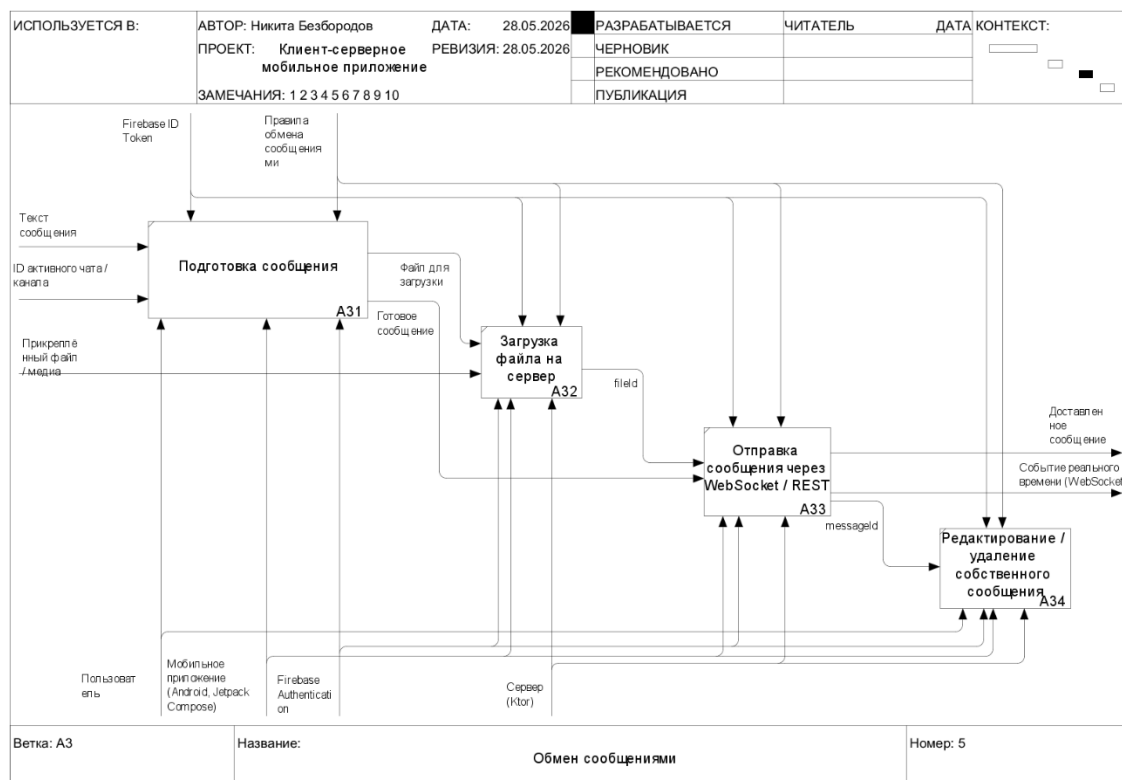


Рисунок 2.3.5 – Декомпозиция процесса «Обмен сообщениями»

На уровне A4 представлена декомпозиция блока «Аудио- и видеозвонки». Он состоит из трёх подпроцессов: «Инициирование звонка» — формирование объекта CallSignal с уникальным callId, идентификаторами участников и флагом isVideo; «Обмен сигнальными данными WebRTC через WebSocket» — сервер Ktor выступает signaling-узлом и пересылает SDP-офферы/ответы и ICE-кандидаты между сторонами по уже установленному WebSocket-каналу; «Установление P2P-соединения и медиапоток» — после согласования параметров клиенты устанавливают прямое peer-to-peer соединение, по которому идут аудио- и видеоданные без участия сервера. Доставка сообщений в реальном времени реализована по той же WebSocket-инфраструктуре в рамках процесса A3.

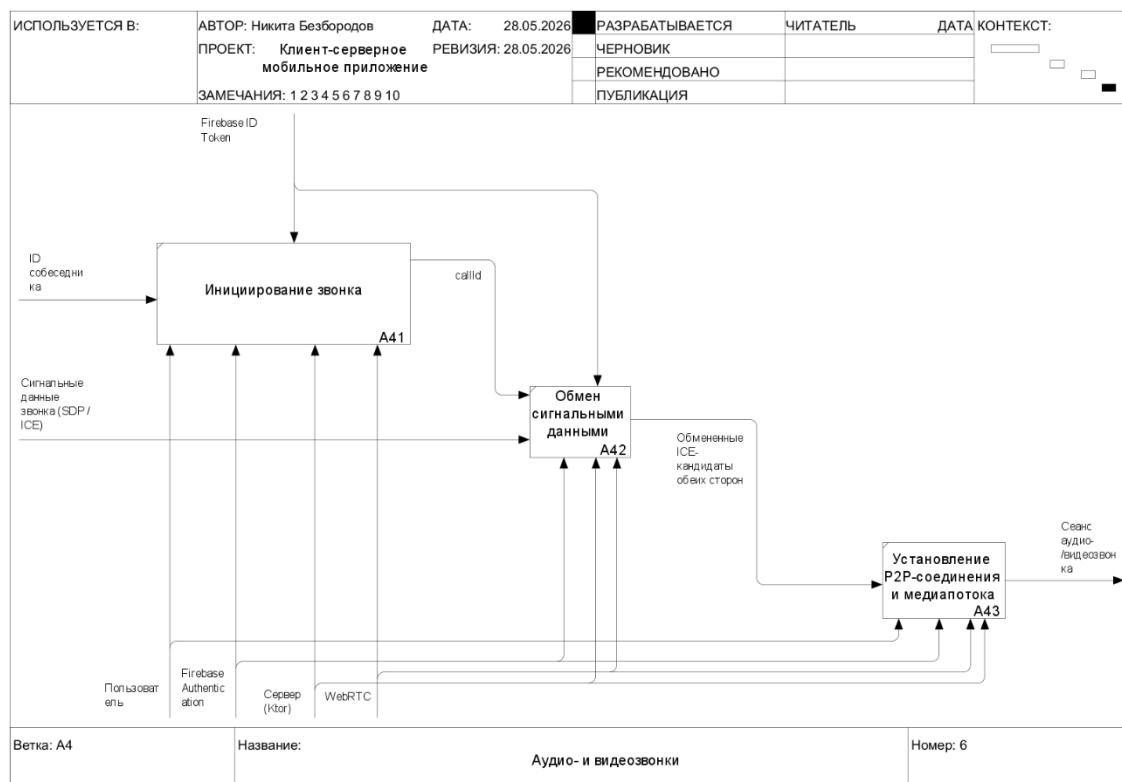


Рисунок 2.3.6 – Декомпозиция процесса «Аудио- и видеозвонки»

Диаграмма классов отражает, как сущности доменного слоя взаимодействуют между собой, позволяя понять структуру и логику работы мессенджера WhatsMAX. Все классы расположены в пакете `com.whatsmax.domain.model`.

Пользователь (User) является центральной сущностью системы. Он создаёт чаты и каналы, участвует в обмене сообщениями, оставляет комментарии под публикациями каналов и инициирует аудио-/видеозвонки. Каждый чат (Chat) может иметь тип

DIRECT (личный 1-на-1) или GROUP (групповой), содержит список участников `ChatMember` и хранит ссылку на последнее сообщение `lastMessage`. Сообщение (Message) принадлежит конкретному чату и отправителю, имеет тип из перечисления

`MessageType` (TEXT, IMAGE, FILE, AUDIO, VIDEO, VOICE, CALL), может содержать рефлексивную ссылку `replyToId` на другое сообщение для функции `reply`, а также флаги `isEdited` и `isDeleted` для операций редактирования и удаления.

Канал (Channel) описывает поток вещания с уникальным handle, владельцем ownerId и счётчиком подписчиков membersCount.

Публикации канала представлены классом ChannelMessage с дополнительными полями views (количество просмотров) и commentsCount. Комментарии под публикациями описаны классом ChannelComment. Звонок представлен классом CallSignal, содержащим signaling-данные WebRTC (SDP и ICE-кандидаты). Файлы, прикреплённые к сообщениям, описаны классом FileInfo с метаданными о размере, MIME-типе и ссылкой на скачивание.

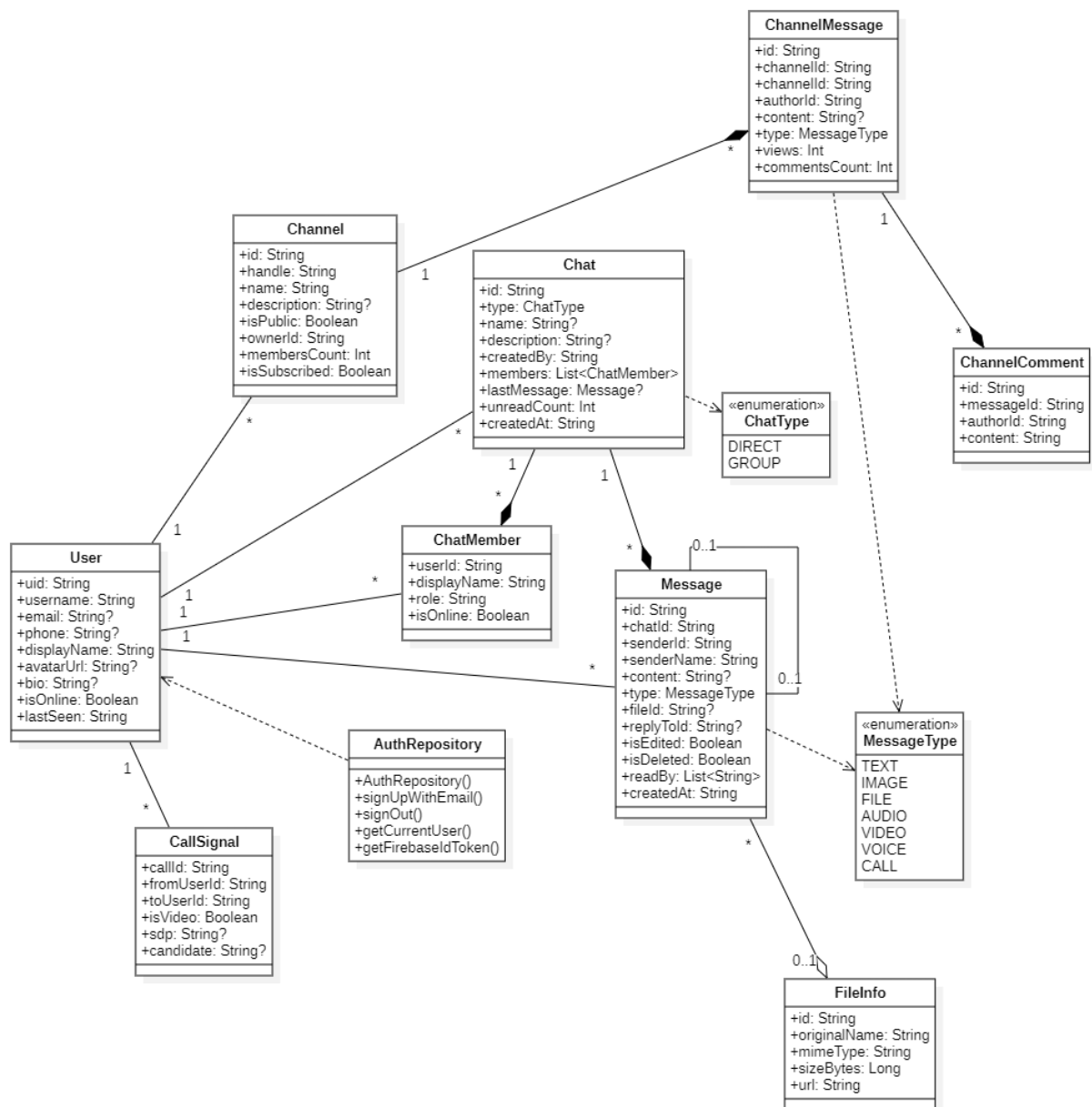


Рисунок 2.3.7 – Диаграмма классов

Таким образом, все сущности приложения логически связаны между собой: пользователи создают чаты и каналы, обмениваются сообщениями и комментируют публикации, сообщения могут ссылаться на файлы и на другие сообщения через механизм reply, а звонки устанавливаются по сигнальным данным WebRTC с маршрутизацией через серверную часть.

2.3.2 Проектирование базы данных (логическая модель БД, описание каждой таблицы)

1. Таблица users

Содержит информацию о зарегистрированных пользователях. Первичным ключом является Firebase UID, полученный после регистрации через Firebase Authentication.

- uid — VARCHAR(128), первичный ключ, уникальный идентификатор пользователя (Firebase UID) ;
- username — VARCHAR(64), уникальный публичный идентификатор (@username) ;
- email — VARCHAR(255), nullable, email пользователя;
- phone — VARCHAR(32), nullable, телефон пользователя;
- display_name — VARCHAR(128), отображаемое имя;
- avatar_url — TEXT, nullable, ссылка на аватар;
- bio — VARCHAR(500), nullable, текст «О себе»;
- is_online — BOOLEAN, флаг присутствия в сети;
- last_seen — TIMESTAMP, время последней активности;
- created_at — TIMESTAMP, дата регистрации;
- fcm_token — TEXT, nullable, токен устройства для push-уведомлений.

2. Таблица chats

Содержит информацию о личных и групповых чатах.

- id — UUID, первичный ключ, генерируется автоматически;
- type — VARCHAR(16), тип чата (direct или group) ;
- name — VARCHAR(128), nullable, название группового чата;

- avatar_url — TEXT, nullable, аватар чата;
- description — TEXT, nullable, описание чата;
- created_by — VARCHAR(128), внешний ключ на users.uid, создатель чата;
- created_at, updated_at — TIMESTAMP.

3. Таблица chat_members

Связка пользователей и чатов (отношение «многие-ко-многим»).

- chat_id — UUID, внешний ключ на chats.id;
- user_id — VARCHAR(128), внешний ключ на users.uid;
- role — VARCHAR(16), роль в чате (member, admin) ;
- joined_at — TIMESTAMP, дата вступления;
- is_deleted — BOOLEAN, флаг выхода из чата;
- Составной первичный ключ — (chat_id, user_id).

4. Таблица messages

Содержит сообщения личных и групповых чатов.

- id — UUID, первичный ключ;
- chat_id — UUID, внешний ключ на chats.id;
- sender_id — VARCHAR(128), внешний ключ на users.uid, отправитель;
- content — TEXT, nullable, текст сообщения;
- type — VARCHAR(16), тип содержимого (text, image, file, audio, video, voice, call) ;
- file_id — UUID, nullable, внешний ключ на files.id;
- reply_to_id — UUID, nullable, ссылка на цитируемое сообщение;
- is_edited, is_deleted — BOOLEAN, флаги операций редактирования и логического удаления;
- created_at, edited_at — TIMESTAMP;
- duration_ms — BIGINT, nullable, длительность голосового/видеосообщения;

– waveform — TEXT, nullable, форма звуковой волны для voice-сообщений;

– Индексы по (chat_id, created_at) и file_id.

5. Таблица message_read_status

Содержит отметки о прочтении сообщений каждым участником чата.

– message_id — UUID, внешний ключ на messages.id;

– user_id — VARCHAR(128), внешний ключ на users.uid;

– read_at — TIMESTAMP, момент прочтения;

– Составной первичный ключ — (message_id, user_id).

6. Таблица channels

Содержит каналы вещания (по аналогии с Telegram).

– id — UUID, первичный ключ;

– handle — VARCHAR(64), уникальный публичный handle (@channel);

– name — VARCHAR(128), название канала;

– description — TEXT, nullable;

– avatar_url — TEXT, nullable;

– is_public — BOOLEAN, флаг открытости канала;

– owner_id — VARCHAR(128), внешний ключ на users.uid, владелец;

– members_count — INT, число подписчиков;

– created_at — TIMESTAMP.

7. Таблица channel_members

Связка подписок пользователей на каналы.

– channel_id — UUID, внешний ключ на channels.id;

– user_id — VARCHAR(128), внешний ключ на users.uid;

– role — VARCHAR(16), роль (subscriber, admin) ;

– joined_at — TIMESTAMP;

– is_muted — BOOLEAN, флаг отключения уведомлений;

– Составной первичный ключ — (channel_id, user_id).

8. Таблица channel_messages

Содержит публикации в каналах.

- id — UUID, первичный ключ;
- channel_id — UUID, внешний ключ на channels.id;
- author_id — VARCHAR(128), внешний ключ на users.uid, автор публикации;
- content — TEXT, nullable;
- type — VARCHAR(16), тип содержимого;
- file_id — UUID, nullable, внешний ключ на files.id;
- views — INT, счётчик просмотров;
- is_edited, is_deleted — BOOLEAN;
- created_at — TIMESTAMP;
- duration_ms, waveform — для голосовых/видеопубликаций;
- Индексы по (channel_id, created_at) и file_id.

9. Таблица channel_comments

Комментарии под публикациями каналов.

- id — UUID, первичный ключ;
- message_id — UUID, внешний ключ на channel_messages.id;
- author_id — VARCHAR(128), внешний ключ на users.uid;
- content — TEXT, текст комментария;
- created_at — TIMESTAMP.

10. Таблица channel_message_views

Уникальные просмотры публикаций каналов каждым пользователем.

- message_id — UUID, внешний ключ на channel_messages.id;
- user_id — VARCHAR(128), внешний ключ на users.uid;
- viewed_at — TIMESTAMP;
- Составной первичный ключ — (message_id, user_id).

11. Таблица files

Метаданные файлов, загруженных через эндпоинт /files;

- id — UUID, первичный ключ;
- original_name — VARCHAR(255), оригинальное имя файла;

- - stored_name — VARCHAR(255), имя файла в хранилище
- mime_type — VARCHAR(128);
- size_bytes — BIGINT, размер в байтах;
- uploaded_by — VARCHAR(128), внешний ключ на users.uidv
- uploaded_at — TIMESTAMP;
- sha256 — VARCHAR(64), nullable, хэш для дедупликации;
- object_key, thumb_object_key — VARCHAR(255), nullable, ключи в объектном хранилище.

12. Таблица reactions

Универсальная таблица реакций (эмодзи) на разные сущности: сообщения, публикации, комментарии.

- id — UUID, первичный ключ;
- entity_type — VARCHAR(16), тип сущности (message, post, comment);
- entity_id — UUID, идентификатор сущности;
- user_id — VARCHAR(128), внешний ключ на users.uid;
- emoji — VARCHAR(8), сама эмодзи-реакция;
- created_at — TIMESTAMP;
- Уникальный индекс — (entity_type, entity_id, user_id), чтобы один пользователь не мог оставить две реакции одного типа на одну сущность.

Логическая модель базы данных WhatsMAX состоит из 12 связанных таблиц, реализованных через ORM Exposed на стороне сервера Ktor. Центральной сущностью является таблица users, на которую ссылаются практически все остальные таблицы через Firebase UID. Чаты и каналы реализованы как самостоятельные сущности (chats, channels) с собственными связками участников (chat_members, channel_members) и публикациями (messages, channel_messages). Файлы вынесены в отдельную таблицу files, что позволяет переиспользовать загруженные ресурсы и осуществлять дедупликацию по SHA-256.

Универсальная таблица reactions спроектирована полиморфно через пару полей (entity_type, entity_id), что позволяет ставить эмодзи-реакции на сообщения чатов, публикации каналов и комментарии без дублирования таблиц.

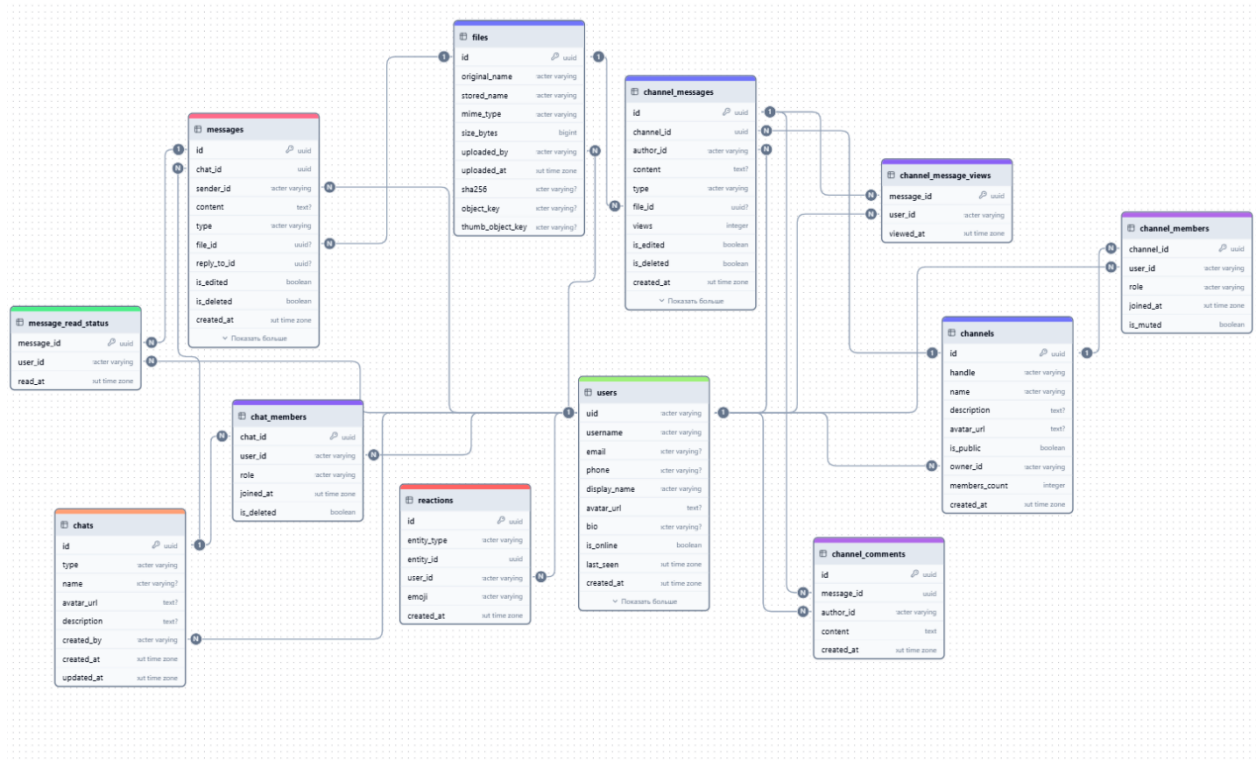


Рисунок 2.3.8 – Логическая модель базы данных

ГЛАВА 3. РАЗРАБОТКА МОБИЛЬНОГО ПРИЛОЖЕНИЯ

3.1. Разработка клиентской части мобильного приложения

Авторизация и регистрация. Функции регистрации и входа реализованы в файлах `LoginViewModel.kt` и `RegisterViewModel.kt`, которые вызывают use case'ы `SignUpWithEmailUseCase` и `SignInWithEmailUseCase` из `AuthUseCases.kt`. Пользователь вводит email, пароль, имя пользователя и отображаемое имя в текстовые поля, после чего данные проходят проверку: пароль должен содержать не менее шести символов, username – не менее трёх символов, email – иметь корректный формат, пароль и его подтверждение – совпадать. При регистрации сначала вызывается `firebaseAuth.createUserWithEmailAndPassword()`, а затем создаётся профиль на бэкенде через `apiService.registerUser(CreateUserRequest)`; при входе после успешной аутентификации Firebase данные профиля загружаются через `apiService.getMe()`. Firebase ID-токен хранится самим Firebase SDK и автоматически добавляется в каждый последующий запрос. Состояние экранов авторизации обновляется в потоках данных `LoginUiState` и `RegisterUiState`, что позволяет интерфейсу реагировать на изменения. В случае ошибки пользователь видит понятное сообщение.

Просмотр и поиск списка чатов. Главный экран реализован в файле `HomeScreen.kt`, который подписывается на поток данных `HomeUiState`, содержащий список чатов, текущего пользователя, поисковый запрос, список найденных пользователей и флаг загрузки. Функция `loadData()` в `HomeViewModel.kt` вызывает `GetChatsUseCase` из `ChatUseCases.kt` и получает список диалогов и групп с последним сообщением и счётчиком непрочитанных. Список чатов отображается в виде карточек, каждая из

которых содержит аватар, название, текст последнего сообщения, время отправки и счётчик непрочитанных сообщений. Поисковая строка через `SearchUsersUseCase` запрашивает пользователей на сервере (при длине запроса от двух символов), а при нажатии на найденного пользователя вызывается `CreateDirectChatUseCase`, и приложение переходит на экран перепис-

ки. Входящие сообщения в реальном времени обрабатываются методом `observeWebSocketMessages()`, который слушает поток `WebSocketRepository`.— `observeEvents()` и обновляет последнее сообщение и счётчик непрочитанных в нужном чате. Во время загрузки отображается ин-дикатор, при отсутствии чатов – сообщение «Нет чатов», при ошибке загрузки – кнопка «Обновить».

Управление чатами и каналами. Создание группового чата реализовано в `HomeScreen.kt` через диалог, в котором пользователь вводит название группы и выбирает участников из списка контактов. После заполнения всех полей пользователь нажимает «Создать», что вызывает `createGroupChat` в `HomeViewModel.kt`. Эта функция через `CreateGroupChatUseCase` отправляет данные на сервер, и после успешного сохранения возвращается на главный экран, где новый чат отображается в списке. Создание канала реализовано аналогично в `ChannelListScreen.kt`, где пользователь задаёт уникальный хэндл (`@channel_name`), название, описание и флаг публичности; сохранение вызывает `CreateChannelUseCase` из `ChannelUseCases.kt`. Редактирование канала доступно через `UpdateChannelUseCase` – название и описание сохраняются на сервере и сразу отображаются в списке. Удаление доступно через выпадающее меню, которое открывается по нажатию на три точки на карточке: при выборе пункта «Удалить» вызывается `DeleteChatUseCase` либо `DeleteChannelUseCase` из соответствующего `ViewModel`, и сущность исчезает из списка. Дополнительно для чатов реализована функция «выйти из чата» через `RemoveMemberFromChatUseCase`, которая удаляет только текущего пользователя из участников, не затрагивая чат целиком.

Отправка и получение сообщений. Экран переписки реализован в файле `ChatScreen.kt`. Пользователь видит ленту сообщений и поле ввода в нижней части экрана. При наборе текста активной становится кнопка отправки, при нажатии на которую вызывается `SendMessageUseCase` из `MessageUseCases.kt`. Этот use case через `MessageRepository` отправляет сообщение на сервер по REST (POST `/chats/{chatId}/messages`), после чего сервер

рассылает его всем участникам чата через WebSocket. Входящие сообщения принимаются через `observeMessages(chatId)` – функцию, возвращающую `Flow<Message>` поверх `WebSocketClient.kt`, и автоматически добавляются в ленту в реальном времени. Поддерживается семь типов сообщений (TEXT, IMAGE, FILE, AUDIO, VIDEO, VOICE, CALL): фотографии и файлы прикрепляются через кнопку «скрепка», голосовые сообщения записываются через `VoiceRecorder.kt` при удержании кнопки микрофона и воспроизводятся через `VoiceMessagePlayer.kt`. Долгое нажатие на сообщение открывает контекстное меню с действиями «Ответить», «Редактировать» и «Удалить», которые вызывают `EditMessageUseCase` и `DeleteMessageUseCase` соответственно. Прочтение фиксируется через `MarkAsReadUseCase`.

Просмотр канала и реакции. Экран канала реализован в файле `ChannelDetailScreen.kt`. При открытии загружаются данные через `GetChannelMessagesUseCase`, который возвращает список постов с количеством просмотров и комментариев. Каждый пост отображается с автором, временем публикации, содержимым и счётчиком просмотров. Под каждым постом доступны реакции (эмодзи), которые добавляются через `SetReactionUseCase` из `ReactionUseCases.kt`, а сводка возвращается в виде `ReactionSummary` с картой `reactions: Map<String, Int>` и текущей реакцией пользователя `myReaction`. Длина полосы прогресса в сводке пропорциональна доле каждого эмодзи от общего числа реакций. Комментарии к постам загружаются через `GetCommentsUseCase` и добавляются через `AddCommentUseCase`. Данные обновляются в реальном времени через WebSocket, что гарантирует их актуальность.

Тёмная тема. Поддержка светлой и тёмной темы реализована в файле `Theme.kt` через `composable`-функцию `WhatsMAXTheme`. Тема автоматически следует системным настройкам Android: параметр `darkTheme` по умолчанию принимает значение из `isSystemInDarkTheme()`, и в зависимости от него `MaterialTheme` применяет одну из двух заранее определённых цветовых схем – `LightColorScheme` или `DarkColorScheme`, построенных на основе Material 3.

Базовая палитра использует фирменный синий цвет (PrimaryBlue = 0xFF0088CC), бирюзовый акцент и нейтральные фоны (BackgroundLight = 0xFFFF5F5F5, BackgroundDark = 0xFF1A1A1A). Для пузырьков сообщений заданы отдельные кастомные цвета через свойства `messageBubbleOwn` и `messageBubbleOther`, которые также переключаются в зависимости от системной темы: собственные сообщения выводятся в зелёных тонах (0xFFDC8C6 / 0xFF056162), чужие – в белом или тёмно-сером (0xFFFFFFFF / 0xFF323739). Индикатор онлайн-статуса (OnlineIndicator = 0xFF4CAF50) одинаков в обеих темах. Благодаря использованию MaterialTheme из Jetpack Compose интерфейс динамически перерисовывается при изменении системной темы без перезапуска приложения.

Сетевой слой. Все REST-запросы к серверу проходят через Retrofit-интерфейс `ApiService.kt`. К каждому запросу автоматически добавляется Firebase ID-токен через `AuthInterceptor.kt`: интерсептор внутри `intercept()` синхронно дёргает `FirebaseAuth.getInstance().currentUser?.getIdToken(false)` и подставляет результат в заголовок `Authorization: Bearer <token>`; при получении ответа с кодом 401 интерсептор автоматически запрашивает свежий токен через `getIdToken(true)` и повторяет запрос. Базовый URL `http://10.0.2.2:8080` (для эмулятора) и WebSocket-URL настроены в DI-модуле; `OkHttpClient` сконфигурирован для работы с JSON через `kotlinx.serialization`. Для real-time связи используется `WebSocketClient.kt` поверх `OkHttp WebSocket`, который публикует входящие события в `Flow<WsEvent>` через `WebSocketRepository`.

Таким образом, в клиентской части приложения реализованы все основные функции интерфейса: регистрация и авторизация, создание, редактирование и удаление чатов и каналов, отправка и получение сообщений в реальном времени, голосовые и видеозвонки, просмотр постов канала с реакциями и комментариями, поиск пользователей, поддержка тёмной темы и корректная обработка ошибок. Каждая функция реализована в соответствующем

файле и использует UseCase для выполнения бизнес-логики в рамках Clean Architecture.

3.2. Разработка серверной части мобильного приложения

3.2.1 Описание основной логики сервера

Серверная часть разработана на языке Kotlin с использованием фреймворка Ktor 2.3.7. Сервер реализует REST API и WebSocket для обработки запросов от мобильного клиента, включая авторизацию пользователей через Firebase, управление чатами и каналами, отправку и получение сообщений в реальном времени, а также управление файлами и реакциями.

Структура серверного приложения. Сервер организован в соответствии с модульной архитектурой по слоям Clean Architecture (domain, data, routes, websocket, plugins, utils). Основные файлы и их назначение:

- Application.kt — точка входа, функция Application.module() последовательно вызывает все configure*-функции для инициализации компонентов;
- data/database/DatabaseFactory.kt — управление подключением к PostgreSQL через HikariCP и инициализация таблиц (UsersTable, ChatsTable, ChatMembersTable, MessagesTable, ChannelsTable, ReactionsTable, FilesTable) с помощью Exposed ORM;
- utils/FirebaseAdmin.kt — инициализация Firebase Admin SDK по сервисному ключу firebase-service-account.json;
- набор плагинов в пакете plugins/: CORS.kt, Serialization.kt, StatusPages.kt, Authentication.kt, WebSockets.kt, RateLimit.kt, Database.kt, Routing.kt — каждый отвечает за настройку своего модуля Ktor;
- обработчики HTTP-запросов: AuthRoutes.kt, UserRoutes.kt, ChatRoutes.kt, MessageRoutes.kt, ChannelRoutes.kt, ReactionRoutes.kt, FileRoutes.kt, WebSocketRoutes.kt.

Авторизация через Firebase. Аутентификация реализована не отдельным эндпоинтом, а на уровне Ktor-плагина Authentication (файл plugins/Authentication.kt) через схему bearer("firebase"), что позволяет защитить любой маршрут одной директивой authenticate("firebase") { ... }. При по-

лучении запроса с заголовком `Authorization: Bearer <token>` плагин извлекает Firebase ID-токен, передаёт его в `Fire-baseAuth.getInstance().verifyIdToken(token, true)` и при успешной проверке формирует объект `FirebasePrincipal(uid, email, name)`. Чтобы не дёргать Fire-base на каждый запрос, реализован in-memory кэш `tokenCache: ConcurrentHashMap<String, CachedPrincipal>` с TTL до 5 минут (но не дольше времени жизни самого токена). Собственный токен сервер не выпускает – во всех за-просах используется тот же Firebase ID-токен. Создание профиля в БД выполняется отдельным защищённым эндпоинтом `POST /auth/register` (файл `AuthRoutes.kt`), который принимает `username` и `displayName`, валидирует их через `Validation.kt` и сохраняет нового пользователя через `UserRepository.createUser(uid, request)`. Если пользователь с таким `uid` уже существует, сервер возвращает существующую запись. Дополнительно реализованы эндпоинты `GET /auth/me` (получение текущего профиля), `DELETE /auth/me` (удаление аккаунта) и `POST /auth/sign-out` (отзыв refresh-токенов че-рез `FirebaseAuth.revokeRefreshTokens(uid)` и инвалидация кэша).

Управление чатами. В файле `ChatRoutes.kt` реализованы следующие эндпоинты:

- `POST /chats` — создаёт новый чат (`direct` или `group`); для прямого чата сначала ищется существующий через `findDirectChat()`, чтобы избежать дублей; создатель автоматически добавляется в участники;
- `GET /chats` — возвращает список чатов текущего пользователя через `chatRepository.getUserChats(uid)`;
- `GET /chats/{id}` — возвращает детальную информацию о конкретном чате, предварительно проверив участие через `isUserInChat()`;
- `PUT /chats/{id}` — обновляет название, описание или аватар чата;

- POST /chats/{id}/members и DELETE /chats/{id}/members/{uid} — добавление и удаление участников (только админам);
- GET /chats/{id}/members — список участников чата;
- DELETE /chats/{id} — удаляет чат полностью (только админам).

Все эндпоинты защищены блоком `authenticate("firebase") { ... }`, что гарантирует наличие валидного `FirebasePrincipal` в `call.principal()`. Аналогичный набор маршрутов для каналов реализован в `ChannelRoutes.kt`.

Отправка и получение сообщений. В файле `MessageRoutes.kt` реализованы эндпоинты, сгруппированные под префиксом `/chats/{chatId}/messages`:

- GET /chats/{chatId}/messages?limit=&before= — возвращает историю сообщений с пагинацией (по умолчанию 50 последних);
- POST /chats/{chatId}/messages — принимает `SendMessageRequest` (тип, содержимое, опционально `fileId`, `replyToId`, метаданные голосового сообщения), сохраняет сообщение через `messageRepository.sendMessage()` и сразу же рассылает событие `WsEvent(NEW_MESSAGE, payload)` всем участникам через `WebSocketManager.sendToUsers()`. Эндпоинт ограничен по частоте через `rateLimit(RateLimitName("messages"))`;
- PUT /chats/{chatId}/messages/{msgId} — редактирует содержимое сообщения (только автору);
- DELETE /chats/{chatId}/messages/{msgId} — удаляет сообщение (только автору);
- POST /chats/{chatId}/messages/{msgId}/read — отмечает сообщение как прочитанное, добавляя `uid` пользователя `read_by`.

Перед каждой операцией проверяется участие пользователя в чате через `chatRepository.isUserInChat(chatId, principal.uid)`, иначе выбрасывается `SecurityException`.

Обработка ошибок и CORS. В файле `plugins/CORS.kt` настроены CORS-заголовки: список разрешённых хостов читается из конфигурации `cors.allowedHosts` (при пустом значении — `anyHost()` с предупреждением в ло-

ге), разрешены методы GET, POST, PUT, DELETE, PATCH, OPTIONS и заголовки Authorization, Content-Type. В файле `plugins/StatusPages.kt` глобальный обработчик исключений перехватывает типовые ошибки и возвращает структурированный `ErrorResponse(error, code)` с соответствующим HTTP-статусом: `IllegalArgumentException` — 400, `SecurityException` — 403, `NoSuchElementException` — 404, любое прочее `Throwable` - 500 (в development-режиме с детальным сообщением, в production — обобщённое). Отдельно обрабатывается статус 401, возвращая клиенту понятное сообщение «Unauthorized. Provide valid Firebase token.» — это срабатывает, когда Authentication-плагин не смог верифицировать Firebase ID-токен.

3.2.2 Подключение базы данных к серверу

В качестве базы данных используется PostgreSQL. Подключение к базе данных реализовано в файле `data/database/DatabaseFactory.kt`. При инициализации сервера (через `configureDatabase()` из `plugins/Database.kt`) загружается JDBC-драйвер PostgreSQL (`org.postgresql.Driver`) и устанавливается соединение по URL `jdbc:postgresql://localhost:5432/whatsmax` с указанием имени пользователя (`postgres`) и пароля, заданных в файле конфигурации `application.conf`. Для управления пулом соединений используется HikariCP с размером пула до 30 одновременных соединений (увеличен с дефолтных 10 после оптимизации N+1 проблем при параллельной нагрузке чата), уровнем изоляции `TRANSACTION_REPEATABLE_READ` и отключённым `auto-commit`. Все параметры подключения могут быть переопределены через переменные окружения `DATABASE_URL`, `DATABASE_USER`, `DATABASE_PASSWORD`, что упрощает деплой в разные среды.

Создание таблиц. При первом запуске сервера автоматически создаются все необходимые таблицы через вызов `SchemaUtils.createMissingTablesAndColumns(...)` из ORM Exposed, который сравнивает текущую схему БД с описанием таблиц в коде и добавляет недостающие. Основные таблицы:

- users — содержит поля: uid (первичный ключ, Firebase UID), username (уникальный индекс), email, phone, display_name, avatar_url, bio, is_online, last_seen, created_at, fcm_token;
- chats — содержит id (первичный ключ, UUID), type (direct или group), name, description, avatar_url, created_by (внешний ключ на users.uid), created_at, updated_at;
- chat_members — связующая таблица «многие ко многим» с составным первичным ключом (chat_id, user_id), а также полями role (admin / member), joined_at, is_deleted;
- messages — содержит id (первичный ключ, UUID), chat_id (внешний ключ на chats.id), sender_id (внешний ключ на users.uid), content, type (text / image / file / audio / video / voice / call), file_id (внешний ключ на files.id), reply_to_id, is_edited, is_deleted, created_at, edited_at, duration_ms, waveform; для оптимизации построены индексы по (chat_id, created_at) и file_id;
- message_read_status — фиксирует факт прочтения сообщения каждым участником, составной ключ (message_id, user_id) и поле read_at;
- channels, channel_members, channel_messages, channel_comments, channel_message_views — аналогичный набор таблиц для функционала каналов (broadcast-сообщения с просмотрами и комментариями);
- files — метаданные загруженных вложений (имя, MIME-тип, размер, путь в MinIO, миниатюра);
- reactions — реакции пользователей на сообщения и посты каналов.

Дополнительно при инициализации создаётся расширение pg_trgm и GIN-индексы по полям users.username, users.display_name, channels.handle, channels.name для быстрого нечувствительного к регистру поиска по подстроке. Также при старте сервер сбрасывает флаг is_online = false для всех пользователей, чтобы не оставались «зависшие» онлайны после перезапуска.

Выполнение SQL-запросов. В отличие от ручной работы с JDBC, запросы к базе выполняются через ORM Exposed, что обеспечивает типобезопасный DSL и автоматическое экранирование параметров. В

DatabaseFactory.kt реализована suspend-функция-обёртка `dbQuery { block }`, которая внутри вызывает `newSuspendedTransaction(Dispatchers.IO) { block() }` — это переносит каждую транзакцию в I/O-диспетчер корутин, не блокируя основной поток сервера. Все репозитории слоя данных (`UserRepositoryImpl`, `ChatRepositoryImpl`, `MessageRepositoryImpl`, `ChannelRepositoryImpl`, `ReactionRepositoryImpl`, `FileRepositoryImpl`) используют эту функцию для взаимодействия с базой. Например, при создании группового чата выполняются вставка записи в таблицу `chats` и последующая массовая вставка участников в `chat_members` в рамках одной транзакции — это гарантирует, что чат либо будет создан полностью с участниками, либо не создан вовсе. При получении списка чатов пользователя выполняется JOIN-запрос между `chats`, `chat_members`, `messages` (для получения последнего сообщения) и подзапрос к `message_read_status` для подсчёта непрочитанных сообщений по каждому чату.

3.2.3 Подключение сервера к клиентской части мобильного приложения

Взаимодействие между клиентской и серверной частями осуществляется по двум протоколам: REST API поверх HTTP для запросов-ответов и WebSocket для двусторонней передачи событий в реальном времени. Клиент отправляет запросы на сервер, а сервер обрабатывает их и возвращает ответы в формате JSON; параллельно через постоянное WebSocket-соединение клиент получает push-обновления (новые сообщения, изменения статусов, сигналы звонков).

Настройка адреса сервера на клиенте. Базовый URL не зашит в исходный код в виде строковой константы, а определён как `buildConfigField` в файле `android/app/build.gradle.kts`: для конфигураций `debug` и `release` задаются поля `BASE_URL = "http://10.0.2.2:8080"` и `WS_URL = "ws://10.0.2.2:8080/ws"`, которые после сборки попадают в автогенерируемый класс `BuildConfig`. В Hilt-модуле `di/NetworkModule.kt` функция `provideRetrofit()` строит Retrofit-инстанс с `baseUrl(BuildConfig.BASE_URL + "/")`, а `provideWebSocketClient()`

инициализирует `WebSocketClient(okHttpClient, BuildConfig.WS_URL)`. Адрес `10.0.2.2` — это специальный `alias` Android-эмулятора, указывающий на `localhost` хост-машины (где запущен Ktor-сервер на порту 8080, заданном в `application.conf`). При запуске на физическом устройстве в `build.gradle.kts` необходимо указать IP-адрес компьютера в локальной сети, на котором запущен сервер.

Формат обмена данными. Все данные между клиентом и сервером передаются в формате JSON. На клиенте сериализация и десериализация выполняются библиотекой `kotlinx.serialization` через `Retrofit2-Kotlinx-Serialization-Converter`; экземпляр `Json` настроен с параметрами `ignoreUnknownKeys = true`, `isLenient = true`, `encodeDefaults = true`. На сервере подключён плагин `ContentNegotiation` с `ktor-serialization-kotlinx-json` (файл `plugins/Serialization.kt`), причём дополнительно включена стратегия именования `JsonNamingStrategy.SnakeCase`, которая автоматически конвертирует поля Kotlin-классов (`camelCase`) в `snake_case` при сериализации и обратно при десериализации — это позволяет именовать столбцы базы данных в стиле PostgreSQL, не загромождая код аннотациями `@SerialName`. Все DTO-классы (`UserDto`, `ChatDto`, `MessageDto`, `ChannelDto`, `CreateChatRequest`, `SendMessageRequest` и др.) размечены аннотацией `@Serializable` для автоматической конвертации.

Процесс авторизации. При входе или регистрации клиент отправляет `email` и `password` напрямую в `Firebase Authentication SDK` через `firebaseAuth.signInWithEmailAndPassword()` или `createUserWithEmailAndPassword()` и получает `Firebase ID-токен`. В отличие от классической схемы «обменять `Firebase-токен` на собственный `JWT` сервера», в `WhatsMAX` сервер собственный `токен` не выпускает — тот же `Firebase ID-токен` используется во всех последующих запросах. За автоматическую подстановку отвечает `AuthInterceptor.kt`: внутри `intercept()` он синхронно дёргает `FirebaseAuth.getInstance().currentUser?.getIdToken(false)` и добавляет результат в заголовок `Authorization: Bearer <token>` каждого запроса. На сто-

роне сервера плагин Authentication (схема bearer("firebase")) при каждом обращении к защищённому маршруту вызывает `FirebaseAuth.getInstance().verifyIdToken(token, true)`, кэширует результат на 5 минут в `ConcurrentHashMap` и помещает в `call.principal()` объект `FirebasePrincipal(uid, email, name)`. Создание профиля в БД выполняется отдельным защищённым эндпоинтом `POST /auth/register`, который вызывается клиентом сразу после успешной регистрации в Firebase.

Пример взаимодействия при создании чата. Клиент формирует объект `CreateChatRequest` с полями `type` (direct или group), `name`, `memberUids` и optionally `description`. Из `HomeViewModel.kt` вызывается use case `CreateGroupChatUseCase` или `CreateDirectChatUseCase`, который через `ChatRepositoryImpl` дёргает `apiService.createChat(request)` — Retrofit отправляет `POST http://10.0.2.2:8080/chats` с JSON-телом и заголовком `Authorization: Bearer <Firebase ID-токен>`. Сервер в `ChatRoutes.kt` верифицирует токен, для direct-чата сначала ищет существующий через `findDirectChat()`, чтобы избежать дубля, добавляет инициатора в список участников и сохраняет чат и записи в `chat_members` в одной транзакции через `chatRepository.createChat()`. В ответ возвращается `ChatDto` с присвоенным `UUID`, который клиент получает и добавляет в список на главном экране через `loadData()`.

Пример взаимодействия при отправке сообщения. Клиент из `ChatViewModel.kt` вызывает `SendMessageUseCase`, который формирует `SendMessageRequest(content, type, fileId, replyToId)` и отправляет `POST http://10.0.2.2:8080/chats/{chatId}/messages`. Сервер в `MessageRoutes.kt` проверяет авторизацию, наличие пользователя в чате через `chatRepository.isUserInChat(chatId, uid)` (иначе `SecurityException` — 403), применяет ограничение по частоте через `rateLimit(RateLimitName("messages"))` и сохраняет сообщение в таблицу `messages`. Сразу после этого сервер получает список участников через `chatRepository.getChatMembers(chatId)` и рассылает им событие `WsEvent(NEW_MESSAGE, Json.encodeToString(message))` через `WebSocketManager.sendToUsers(members, event)`. Отправитель получает ответ

MessageDto через REST, а все остальные участники чата – через своё открытое WebSocket-соединение, что обеспечивает real-time доставку без polling-а.

Обработка ошибок. При возникновении ошибки на сервере клиент получает соответствующий HTTP-статус и тело ErrorResponse(error, code), сформированное плагином StatusPages.kt (IllegalArgumentException — 400, SecurityException — 403, NoSuchElementException — 404, прочее — 500). На клиенте репозитории преобразуют неуспешные Response от Retrofit в Result.Error(message), а ViewModel'и перехватывают их и обновляют поле error в UiState, которое экран отображает понятным пользователю сообщением. Особым образом обрабатывается код 401: AuthInterceptor сам перехватывает его, принудительно обновляет Firebase ID-токен через getIdToken(forceRefresh = true) и автоматически повторяет запрос; только если и это не помогает, клиент перенаправляет пользователя на экран авторизации.

Синхронизация данных. В отличие от классической схемы с принудительной перезагрузкой списка после каждой операции, в WhatsMAX актуальность данных обеспечивается двумя механизмами. Во-первых, для собственных действий пользователя (отправка сообщения, создание чата) клиент сразу применяет результат REST-ответа к UiState. Во-вторых, для внешних изменений (сообщения от других пользователей, новые подписки) используется постоянное WebSocket-соединение: HomeViewModel в методе observeWebSocketMessages() подписывается на wsRepository.observeEvents(), и при получении события NEW_MESSAGE инкрементирует счётчик непрочитанных в нужном чате, обновляет поле lastMessage и сортирует список по времени последнего сообщения. Принудительный loadData() вызывается только как fallback – если парсинг WebSocket-события завершился ошибкой, либо если пользователь явно делает swipe-to-refresh.

Таким образом, серверная часть обеспечивает надёжное взаимодействие с клиентом через REST API и WebSocket, выполняет авторизацию через Firebase Admin SDK без выдачи собственных токенов, управляет базой

данных PostgreSQL через ORM Exposed и возвращает данные в формате JSON, а клиент корректно обрабатывает ответы, отображает их пользователю и поддерживает актуальное состояние интерфейса в реальном времени.

3.3. Тестирование разработанного приложения, unit-тестирование, описание полученных результатов)

В данном разделе мы демонстрируем, как автоматические unit-тесты подтверждают корректность ключевых операций сервера мессенджера. Тесты написаны на JUnit 4 с использованием `kotlin-test-junit`, инфраструктуры `ktor-server-test-host` и библиотеки моков `MockK 1.13.9`. Все маршруты тестируются изолированно: вместо реальной PostgreSQL и Firebase Admin SDK используется тестовый `testModule()`, который подменяет репозитории моковыми объектами, а аутентификацию — фейковым `bearer`-провайдером, извлекающим `uid` пользователя прямо из значения токена. Это позволяет проверять бизнес-логику маршрутов быстро (вся серия из 22 тестов выполняется за ~4 секунды) и без зависимости от внешней инфраструктуры.

Тесты собраны в файле `backend/src/test/kotlin/com/whatsmax/WhatsMaxApiTest.kt`; вспомогательная функция `runTestApp { mocks -> ... }` из `TestSupport.kt` поднимает Ktor-приложение с пустым конфигом, настраивает моки и регистрирует все маршруты:

- **registerUserSuccess** — проверяет, что POST-запрос на `/auth/register` с уникальным `uid` (Firebase верифицировал токен) и валидным именем пользователя создаёт новую запись в БД. Сервер возвращает статус 201 и сериализованный объект `User`. Mock `userRepository.createUser()` вызывается ровно один раз. Тест пройден;

- **registerExistingUserReturns200** — проверяет идемпотентность регистрации: при повторной регистрации пользователя с тем же Firebase `uid` сервер возвращает статус 200 и существующий профиль, не создавая дубль. Mock `createUser()` не вызывается. Тест пройден;

– **registerInvalidUsernameFails** — проверяет валидацию имени пользователя. При отправке `username = "ab"` (менее 3 символов) сервер возвращает статус 400 с сообщением «Username must be 3-32 characters, letters/digits/underscore only», брошенным из `Validation.validateUsername()`.

Тест пройден;

– **getMeReturnsUser** — проверяет получение профиля текущего пользователя через `GET /auth/me`. Сервер возвращает статус 200 и JSON-объект с полями `uid`, `username`, `displayName`. Тест пройден;

– **getMeForUnknownUserReturns404** — проверяет, что для неизвестного `uid` сервер возвращает статус 404 с сообщением «User not found. Please register first.». Тест пройден;

– **createDirectChatSuccess** — проверяет создание прямого чата через `POST /chats` с `type = "direct"`. Сервер возвращает статус 201, при этом сам инициатор автоматически добавляется в список участников через `(memberUids + principal.uid).distinct()`. Проверка через `slot-<CreateChatRequest>` подтверждает корректность переданных в репозиторий данных. Тест пройден;

– **createDirectChatReturnsExisting** — проверяет, что при попытке создать прямой чат, который уже существует, сервер возвращает существующую запись со статусом 200, а `createChat()` не вызывается. Тест пройден;

– **createGroupChatSuccess** — проверяет создание группового чата через `POST /chats` с `type = "group"`. Сервер возвращает статус 201 и объект `Chat` с полем `type = "group"`. Тест пройден;

– **getChatsReturnsList** — проверяет, что `GET /chats` возвращает массив объектов чатов пользователя со статусом 200. Тест пройден;

– **getChatByIdForNonMemberFails403** — проверяет защиту от доступа к чужому чату: при запросе `GET /chats/{id}` пользователем, не состоящим в чате, сервер выбрасывает `SecurityException("Access denied")` и возвращает статус 403. Тест пройден;

– **sendMessageSuccess** — проверяет отправку сообщения через POST /chats/{chatId}/messages. Сервер сохраняет сообщение, возвращает статус 201 с объектом Message и рассылает событие WsEvent(NEW_MESSAGE, ...) всем участникам чата. Через coVerify подтверждается, что WebSocketManager.sendToUsers() был вызван ровно один раз со списком, содержащим все uid участников. Тест пройден;

– **sendMessageInForeignChatFails403** — проверяет защиту от спама: при попытке отправить сообщение в чат, в котором пользователь не состоит, сервер возвращает 403, а messageRepository.sendMessage() не вызывается. Тест пройден;

– **getMessagesReturnsList** — проверяет получение истории сообщений через GET /chats/{chatId}/messages. Сервер возвращает массив Message со статусом 200 после проверки членства пользователя в чате. Тест пройден;

– **editOwnMessageSuccess** — проверяет редактирование собственного сообщения через PUT /chats/{chatId}/messages/{msgId}. Сервер возвращает обновлённое сообщение со статусом 200 и флагом isEdited = true. Тест пройден;

– **editForeignMessageFails403** — проверяет, что пользователь не может редактировать чужие сообщения: сервер выбрасывает SecurityException("Cannot edit others' messages") и возвращает 403; editMessage() не вызывается. Тест пройден;

– **deleteOwnMessageSuccess** — проверяет удаление собственного сообщения через DELETE /chats/{chatId}/messages/{msgId}. Сервер возвращает статус 204 (No Content) и вызывает messageRepository.deleteMessage(msgId). Тест пройден;

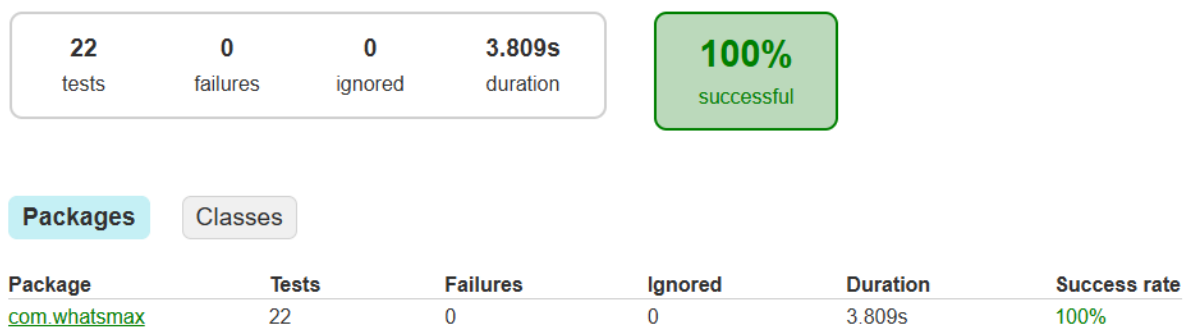
– **unauthorisedAccessFails401** — проверяет защищённость всех ключевых эндпоинтов: запросы на /auth/me, GET /chats, POST /chats, POST /chats/{id}/messages без заголовка Authorization: Bearer ... возвращают статус 401 с сообщением «Unauthorized. Provide valid Firebase token.». Тест пройден;

– **searchUsersByQuerySuccess** — проверяет поиск пользователей через GET /users/search?q=ali. Сервер возвращает массив User со статусом 200, в котором все найденные пользователи содержат подстроку запроса в username. Тест пройден;

– **searchUsersWithoutQueryFails400** — проверяет валидацию параметра поиска: при отсутствии параметра q сервер выбрасывает IllegalArgumentException("Query parameter 'q' is required") и возвращает статус 400. Тест пройден;

– **unknownRouteReturns404** — проверяет глобальный обработчик 404: для несуществующего маршрута /nonexistent сервер возвращает структурированный ErrorResponse(error, code) со статусом 404. Тест пройден.

Test Summary



Generated by [Gradle 8.12](#) at 29 мая 2026 г., 14:37:32

Рисунок 3.3 – Успешное выполнение тестов

Все 22 теста проходят успешно, подтверждая корректность базовой работы API для регистрации пользователей через Firebase, получения профилей, создания и просмотра чатов (прямых и групповых), отправки и редактирования сообщений с рассылкой через WebSocket, поиска пользователей, а также защиты приватных эндпоинтов от неавторизованного доступа.

Приложение готово к эксплуатации и защите курсовой работы.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано клиент-серверное мобильное приложение — мессенджер WhatsMAX.

Приложение реализовано на языке Kotlin с использованием современных технологий: Jetpack Compose для построения пользовательского интерфейса на клиентской стороне и фреймворка Ktor для разработки серверной части. В качестве базы данных использована PostgreSQL в связке с ORM Exposed, обеспечивающая надёжное хранение информации о пользователях, чатах, сообщениях, каналах, файлах и реакциях. Аутентификация пользователей реализована через Firebase Authentication, push-уведомления — через Firebase Cloud Messaging, передача сообщений в реальном времени — через WebSocket, аудио- и видеозвонки — через WebRTC, а хранение медиа-вложений – через S3-совместимое хранилище MinIO.

Все поставленные задачи были выполнены в полном объёме. Проведён анализ предметной области, рассмотрены существующие аналоги (Telegram, WhatsApp, Viber, Signal), на основе которого были выявлены их недостатки и обоснован выбор собственного технологического стека. Сформулированы функциональные и нефункциональные требования к приложению, разработана логическая модель базы данных из двенадцати таблиц, спроектирована архитектура приложения в соответствии с принципами Clean Architecture с разделением на слои domain, data и presentation на клиенте и routes, domain, data, websocket, plugins на сервере.

Разработанное приложение позволяет пользователям регистрироваться и авторизоваться через Firebase, создавать прямые и групповые чаты, отправлять и получать сообщения в реальном времени, прикреплять фотографии, файлы и голосовые сообщения с визуализацией звуковой волны (waveform), отвечать на сообщения, редактировать и удалять собственные сообщения, ставить реакции (эмодзи), просматривать статусы прочтения и индикаторы набора текста. Реализованы каналы по типу Telegram — с публикацией постов, подсчётом просмотров, комментариями и подписками.

Поддерживаются аудио- и видеозвонки на базе WebRTC и push-уведомления через FCM. Реализован поиск пользователей и каналов с использованием trigram-индексов PostgreSQL для быстрого нечувствительного к регистру поиска по подстроке. Поддержана светлая и тёмная тема оформления на основе Material 3, автоматически следующая системным настройкам Android.

Проведённое тестирование подтвердило корректность работы всех функций приложения как на стороне клиента, так и на стороне сервера. Серверная часть покрыта 22 автоматическими unit-тестами (JUnit 4 + MockK + ktor-server-test-host), проверяющими регистрацию пользователей, создание прямых и групповых чатов, отправку сообщений с рассылкой через WebSocket, редактирование и удаление сообщений, поиск пользователей, а также защиту приватных эндпоинтов от неавторизованного доступа — все тесты выполняются за ~4 секунды и проходят успешно. Сервер успешно обрабатывает REST- и WebSocket-запросы, выполняет авторизацию через Firebase Admin SDK с in-memory кэшированием токенов, взаимодействует с базой данных PostgreSQL через ORM Exposed и возвращает данные в формате JSON. Клиент корректно отображает информацию, обрабатывает ошибки и обеспечивает удобный пользовательский интерфейс на основе Jetpack Compose.

Таким образом, цель курсовой работы достигнута. Полученное приложение представляет собой полнофункциональный мессенджер с возможностями, сопоставимыми с коммерческими аналогами, и может быть использовано для организации обмена сообщениями в учебных, корпоративных и личных целях, а также служить базой для дальнейшего развития — добавления end-to-end шифрования, ботов, сторис или интеграции с внешними сервисами.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Митяков, Е. С. Современная разработка мобильных приложений на языке Kotlin : практикум / Е. С. Митяков, М. С. Поздняков. — Москва : РТУ МИРЭА, 2025.
2. Степанов, П. В. Разработка мобильных приложений под Android на языке Kotlin : учебное пособие / П. В. Степанов, Н. А. Приходько, А. И. Запорожских. — Москва : РТУ МИРЭА, 2025. — 889 с. — ISBN 978-5-7339-2596-7.
3. Официальный сайт языка Kotlin. Документация. — URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 12.12.2024).
4. Руководство по созданию приложений под Android с помощью Kotlin и Jetpack Compose. [Metanit.com](https://metanit.com). — URL: <https://metanit.com/kotlin/jetpack/> (дата обращения: 29.05.2025).
5. Официальный сайт фреймворка Ktor. Документация. — URL: <https://ktor.io/docs/welcome.html> (дата обращения: 29.05.2026).
6. Официальный сайт облачного сервиса Google Firebase. Документация. — URL: <https://firebase.google.com/docs?hl=ru> (дата обращения: 29.05.2026).
7. Официальный сайт PostgreSQL. \ Документация. — URL: <https://www.postgresql.org/docs/> (дата обращения: 29.05.2026).
8. Официальный сайт Jetpack Compose. Android Developers. — URL: <https://developer.android.com/jetpack/compose/documentation> (дата обращения: 29.05.2026).

ПРИЛОЖЕНИЯ

Приложение А. Полный листинг мобильного приложения.

Приложение Б. Скриншоты всех экранов мобильного приложения.

Приложение В. Все тестовые сценарии.

Приложение А. Полный листинг мобильного приложения

Клиентская часть:

<https://github.com/BEZBIG/Messenger-WhatsMAX/tree/main/android>

Серверная часть:

<https://github.com/BEZBIG/Messenger-WhatsMAX/tree/main/backend>

Приложение Б. Скриншоты всех экранов мобильного приложения

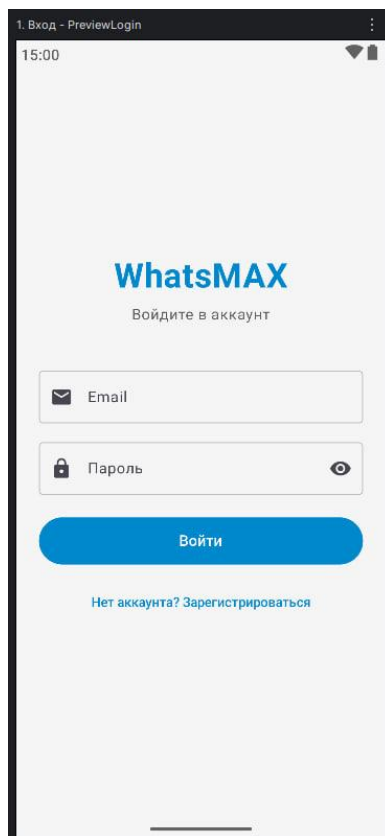


Рисунок 12 – Экран Авторизации

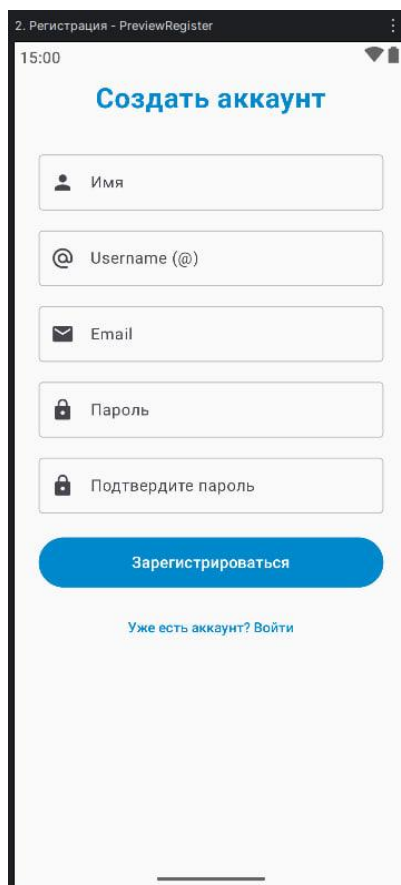


Рисунок 13 – Экран Регистрации

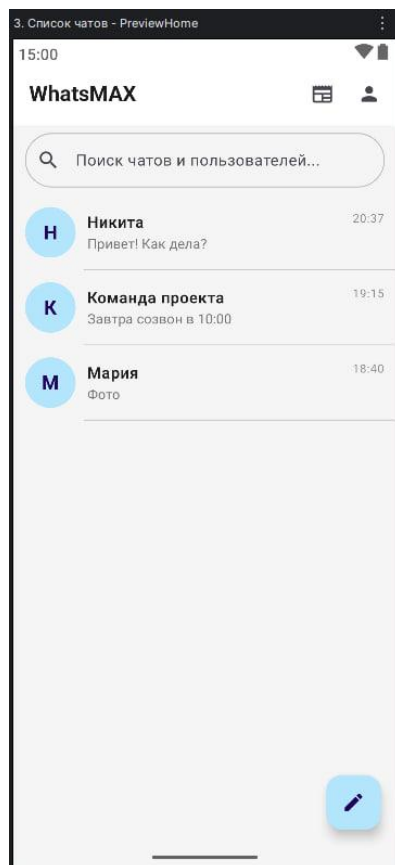


Рисунок 14 – Главный экран

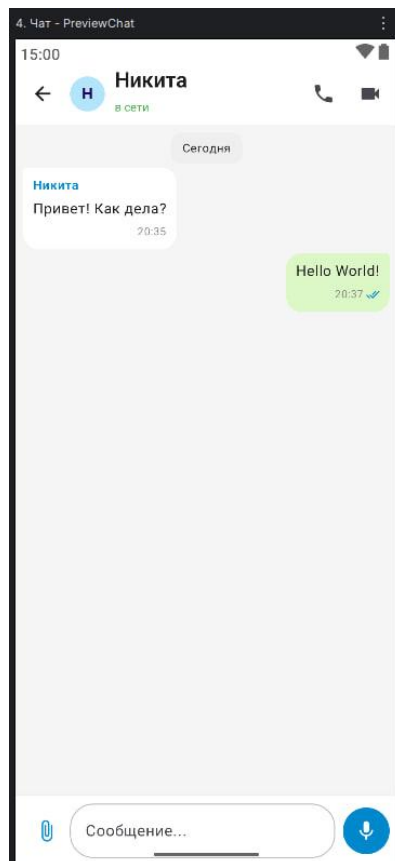


Рисунок 15 – Экран Чата

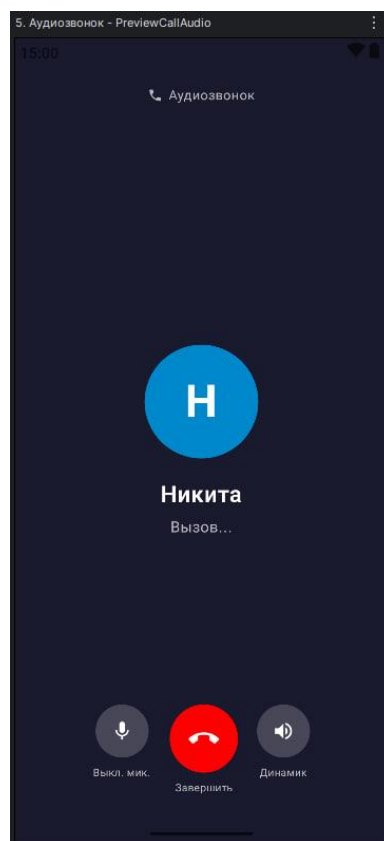


Рисунок 16 – Экран Аудиозвонка

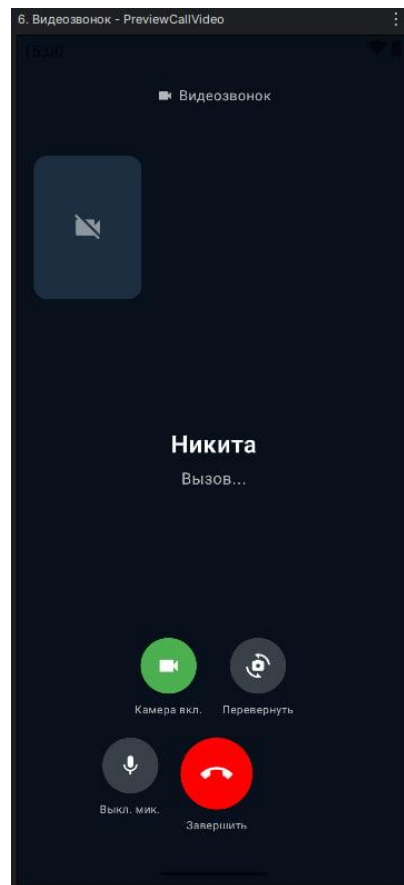


Рисунок 17 – Экран Видеозвонка

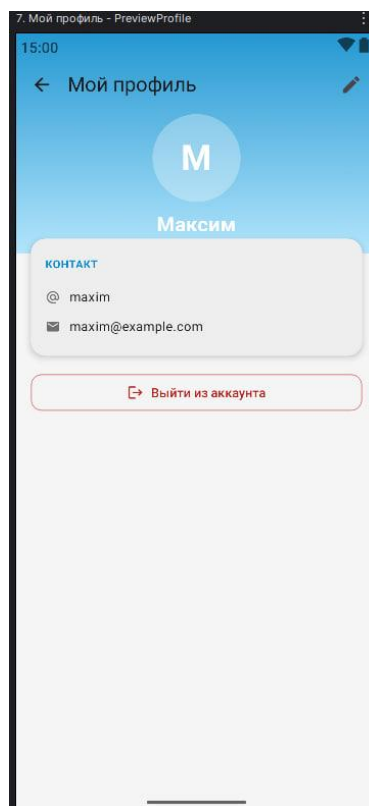


Рисунок 18 – Экран Мой профиль

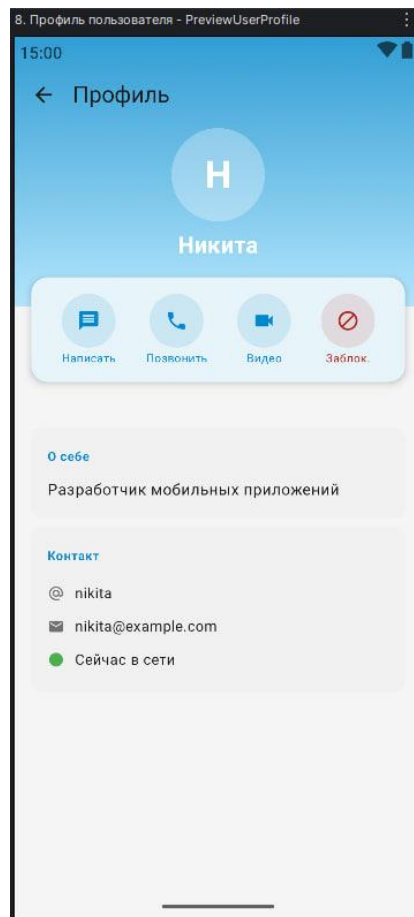


Рисунок 19 – Экран Профиль собеседника

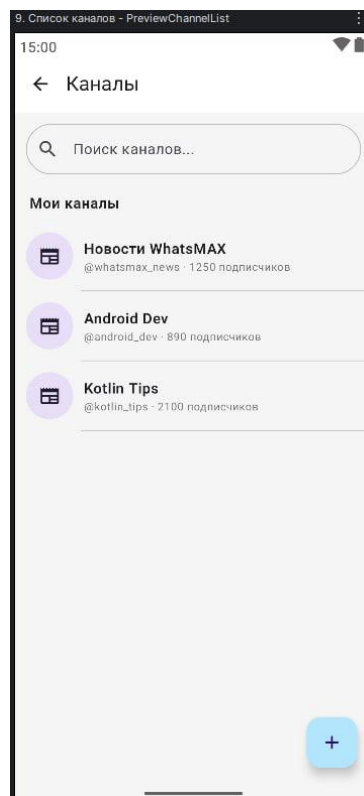


Рисунок 20 – Экран каналов

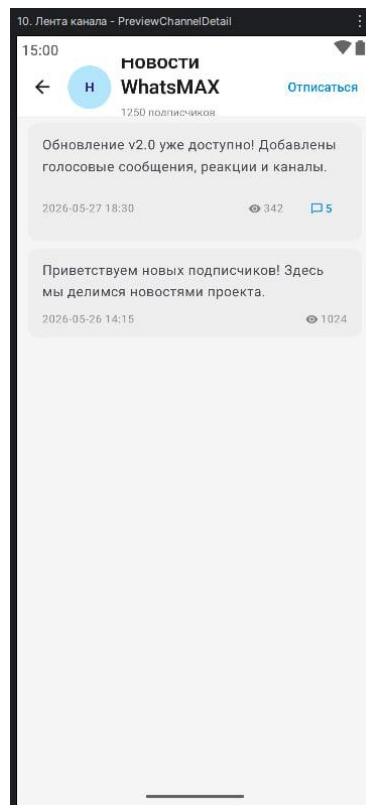


Рисунок 21 – Экран Лента канала

Приложение В. Все тестовые сценарии

Листинг 1. Application.kt

```
/** Базовые smoke-тесты бэкенда (используют testModule без БД/Firebase). */
package com.whatsmax

import io.ktor.client.request.get
import io.ktor.http.HttpStatusCode
import kotlin.test.Test
import kotlin.test.assertEquals

class ApplicationTest {

    @Test
    fun `protected endpoint returns 401 without token`() = runTestApp { _ ->
        val response = client.get("/auth/me")
        assertEquals(HttpStatusCode.Unauthorized, response.status)
    }

    @Test
    fun `unknown route returns 404`() = runTestApp { _ ->
        val response = client.get("/nonexistent")
        assertEquals(HttpStatusCode.NotFound, response.status)
    }
}
```

Листинг 2. TestSupport.kt

```
/** Тестовая инфраструктура: testModule без БД/Firebase + fake auth + моки. */
package com.whatsmax

import com.whatsmax.domain.repositories.*
import com.whatsmax.plugins.FirebasePrincipal
import com.whatsmax.plugins.configureStatusPages
import com.whatsmax.routes.*
import com.whatsmax.utils.StorageService
import com.whatsmax.websocket.WebSocketManager
import io.ktor.client.HttpClient
import io.ktor.client.plugins.contentnegotiation.ContentNegotiation as ClientContentNegotiation
import io.ktor.client.request.HttpRequestBuilder
import io.ktor.client.request.bearerAuth
import io.ktor.serialization.kotlinx.json.json
import io.ktor.server.application.Application
import io.ktor.server.application.install
import io.ktor.server.auth.Authentication
import io.ktor.server.auth.bearer
import io.ktor.server.auth.principal
import io.ktor.server.config.MapApplicationConfig
import io.ktor.server.plugins.contentnegotiation.ContentNegotiation
import io.ktor.server.plugins.defaultheaders.DefaultHeaders
import io.ktor.server.plugins.ratelimit.RateLimit
import io.ktor.server.plugins.ratelimit.RateLimitName
import io.ktor.server.routing.routing
import io.ktor.server.testing.ApplicationTestBuilder
import io.ktor.server.testing.testApplication
import io.mockk.coEvery
import io.mockk.mockk
import kotlinx.serialization.json.Json
import kotlinx.serialization.json.JsonNamingStrategy
```

```

import kotlin.time.Duration.Companion.minutes

/**
 * Фейковый формат токена: только uid (alphanumeric + underscore).
 * Bearer-парсер Ktor требует, чтобы значение было token68 — без пробелов,
 * спецсимволов вроде ":" и "@".
 * Поэтому имя/email в принicipала подставляются константами, а сам токен —
 * это просто uid.
 */
fun fakeToken(uid: String): String = uid

fun HttpRequestBuilder.bearer(token: String) = bearerAuth(token)

/** JSON-кодек для клиента в тестах — повторяет серверную SnakeCase-
стратегию. */
@OptIn(kotlinx.serialization.ExperimentalSerializationApi::class)
val testJson: Json = Json {
    isLenient = true
    ignoreUnknownKeys = true
    encodeDefaults = true
    namingStrategy = JsonNamingStrategy.SnakeCase
}

/**
 * Контейнер с моками для использования в каждом тесте.
 */
class TestMocks(
    val users: UserRepository = mockk(relaxed = false),
    val chats: ChatRepository = mockk(relaxed = false),
    val messages: MessageRepository = mockk(relaxed = false),
    val channels: ChannelRepository = mockk(relaxed = false),
    val files: FileRepository = mockk(relaxed = true),
    val reactions: ReactionRepository = mockk(relaxed = true),
    val ws: WebSocketManager = mockk(relaxed = true),
    val storage: StorageService = mockk(relaxed = true)
)

/**
 * Тестовый модуль приложения: подключает только нужные плагины и регистриру-
ет маршруты
 * с замканными репозиториями. БД и Firebase Admin SDK не задействуются.
 */
@OptIn(kotlinx.serialization.ExperimentalSerializationApi::class)
fun Application.testModule(mocks: TestMocks) {

    install(DefaultHeaders) { header("X-App-Name", "WhatsMAX-Backend-Test") }

    install(ContentNegotiation) {
        json(Json {
            prettyPrint = true
            isLenient = true
            ignoreUnknownKeys = true
            encodeDefaults = true
            namingStrategy = JsonNamingStrategy.SnakeCase
        })
    }

    configureStatusPages()

    install(Authentication) {
        bearer("firebase") {
            authenticate { credential ->
                val uid = credential.token
            }
        }
    }
}

```

```

        if (uid.isNotBlank()) {
            FirebasePrincipal(uid = uid, name = "TestUser", email =
"test@example.com")
        } else null
    }
}

install(RateLimit) {
    val ids = listOf("messages", "reactions", "uploads", "auth")
    ids.forEach { name ->
        register(RateLimitName(name)) {
            rateLimiter(limit = 100_000, refillPeriod = 1.minutes)
            requestKey { call ->
                call.principal<FirebasePrincipal>()?.uid
                ?: call.request.local.remoteHost
            }
        }
    }
}

coEvery { mocks.ws.sendToUsers(any(), any()) } returns Unit

routing {
    authRoutes(mocks.users)
    userRoutes(mocks.users)
    chatRoutes(mocks.chats, mocks.users)
    messageRoutes(mocks.messages, mocks.chats, mocks.files,
mocks.storage, mocks.ws, "/tmp")
}

}

/** Хелпер: настройка клиента в testApplication для работы с JSON в Snake-
Case. */
fun ApplicationTestBuilder.jsonClient(): HttpClient = createClient {
    install(ClientContentNegotiation) {
        json(testJson)
    }
}

}

/**
 * Обёртка над testApplication, которая:
 * - перебивает environment на пустой MapApplicationConfig (иначе Ktor сам
подхватит
 *    основной module() из application.conf и попыбует подключиться к Post-
greSQL);
 * - регистрирует testModule(mocks);
 * - даёт блоку доступ к mocks для настройки coEvery {...}.
 */
fun runTestApp(block: suspend ApplicationTestBuilder.(TestMocks) -> Unit) =
testApplication {
    environment { config = MapApplicationConfig() }
    val mocks = TestMocks()
    application { testModule(mocks) }
    block(mocks)
}
}

```

Листинг 2. WhatsMaxApiTest.kt

```

/** Интеграционные HTTP-тесты ключевых API мессенджера WhatsMAX. */
package com.whatsmax

import com.whatsmax.domain.models.*
import io.ktor.client.call.body

```

```

import io.ktor.client.request.*
import io.ktor.client.statement.bodyAsText
import io.ktor.http.*
import io.mockk.coEvery
import io.mockk.coVerify
import io.mockk.slot
import java.util.UUID
import kotlin.test.Test
import kotlin.test.assertEquals
import kotlin.test.assertTrue

class WhatsMaxApiTest {

    // ——— Тестовые данные ———
    private val aliceUid = "uid_alice"
    private val bobUid   = "uid_bob"
    private val now       = "2026-05-29T10:00:00"

    private fun makeUser(uid: String, username: String = "user_$uid") = User(
        uid = uid, username = username, email = "$username@x.com",
        displayName = username, lastSeen = now, createdAt = now
    )

    private fun makeChat(
        id: UUID = UUID.randomUUID(), type: String = "group",
        createdBy: String = aliceUid, name: String? = "Test chat"
    ) = Chat(
        id = id.toString(), type = type, name = name, createdBy = createdBy,
        createdAt = now, updatedAt = now
    )

    private fun makeMessage(
        chatId: UUID, senderId: String, content: String = "hello",
        id: UUID = UUID.randomUUID()
    ) = Message(
        id = id.toString(), chatId = chatId.toString(), senderId = senderId,
        senderName = "User", content = content, createdAt = now
    )

    // ——— 1. registerUserSuccess ———
    @Test
    fun `registerUserSuccess - 201 при создании нового профиля`() =
runTestApp { mocks ->
        val client = jsonClient()
        coEvery { mocks.users.getUserByUid(aliceUid) } returns null
        coEvery { mocks.users.createUser(aliceUid, any()) } returns
makeUser(aliceUid, "alice")

        val response = client.post("/auth/register") {
            bearer(fakeToken(aliceUid))
            contentType(ContentType.Application.Json)
            setBody(CreateUserRequest(username = "alice", displayName = "Al-
ice"))
        }

        assertEquals(HttpStatusCode.Created, response.status)
        val user: User = response.body()
        assertEquals("alice", user.username)
        coVerify(exactly = 1) { mocks.users.createUser(aliceUid, any()) }
    }

    // ——— 2. registerExistingUserReturns200 ———
    @Test

```

```

    fun `registerExistingUserReturns200 - повторная регистрация возвращает
    существующего`() = runTestApp { mocks ->
        val client = jsonClient()
        val existing = makeUser(aliceUid, "alice")
        coEvery { mocks.users.getUserByUid(aliceUid) } returns existing

        val response = client.post("/auth/register") {
            bearer(fakeToken(aliceUid))
            contentType(ContentType.Application.Json)
            setBody(CreateUserRequest(username = "alice", displayName = "Al-
ice"))
        }

        assertEquals(HttpStatusCode.OK, response.status)
        coVerify(exactly = 0) { mocks.users.createUser(any(), any()) }
    }

    // — 3. registerInvalidUsernameFails —
    @Test
    fun `registerInvalidUsernameFails - 400 для короткого username`() =
    runTestApp { mocks ->
        val client = jsonClient()

        val response = client.post("/auth/register") {
            bearer(fakeToken(aliceUid))
            contentType(ContentType.Application.Json)
            setBody(CreateUserRequest(username = "ab", displayName = "Al-
ice"))
        }

        assertEquals(HttpStatusCode.BadRequest, response.status)
        assertTrue(response.bodyAsText().contains("Username"))
    }

    // — 4. getMeReturnsUser —
    @Test
    fun `getMeReturnsUser - 200 для зарегистрированного пользователя`() =
    runTestApp { mocks ->
        val client = jsonClient()
        coEvery { mocks.users.getUserByUid(aliceUid) } returns
        makeUser(aliceUid, "alice")

        val response = client.get("/auth/me") { bearer(fakeToken(aliceUid)) }

        assertEquals(HttpStatusCode.OK, response.status)
        val user: User = response.body()
        assertEquals("alice", user.username)
    }

    // — 5. getMeForUnknownUserReturns404 —
    @Test
    fun `getMeForUnknownUserReturns404 - 404 если профиль не создан`() =
    runTestApp { mocks ->
        val client = jsonClient()
        coEvery { mocks.users.getUserByUid(aliceUid) } returns null

        val response = client.get("/auth/me") { bearer(fakeToken(aliceUid)) }

        assertEquals(HttpStatusCode.NotFound, response.status)
    }

    // — 6. createDirectChatSuccess —
    @Test

```

```

        fun `createDirectChatSuccess - 201 и автоматическое добавление
инициатора`() = runTestApp { mocks ->
    val client = jsonClient()
    val newChat = makeChat(type = "direct", name = null)
    coEvery { mocks.chats.findDirectChat(aliceUid, bobUid) } returns null
    val captured = slot<CreateChatRequest>()
    coEvery { mocks.chats.createChat(capture(captured), aliceUid) } re-
turns newChat

    val response = client.post("/chats") {
        bearer(fakeToken(aliceUid))
        contentType(ContentType.Application.Json)
        setBody(CreateChatRequest(type = "direct", memberUids = lis-
tOf(bobUid)))
    }

    assertEquals(HttpStatusCode.Created, response.status)
    assertTrue(captured.captured.memberUids.containsAll(listOf(aliceUid,
bobUid)))
}

// — 7. createDirectChatReturnsExisting —————
@Test
fun `createDirectChatReturnsExisting - 200 если direct-чат уже есть`() =
runTestApp { mocks ->
    val client = jsonClient()
    val existing = makeChat(type = "direct", name = null)
    coEvery { mocks.chats.findDirectChat(aliceUid, bobUid) } returns ex-
isting

    val response = client.post("/chats") {
        bearer(fakeToken(aliceUid))
        contentType(ContentType.Application.Json)
        setBody(CreateChatRequest(type = "direct", memberUids = lis-
tOf(bobUid)))
    }

    assertEquals(HttpStatusCode.OK, response.status)
    coVerify(exactly = 0) { mocks.chats.createChat(any(), any()) }
}

// — 8. createGroupChatSuccess —————
@Test
fun `createGroupChatSuccess - 201 для группового чата`() = runTestApp {
mocks ->
    val client = jsonClient()
    val groupChat = makeChat(type = "group", name = "Squad")
    coEvery { mocks.chats.createChat(any(), aliceUid) } returns groupChat

    val response = client.post("/chats") {
        bearer(fakeToken(aliceUid))
        contentType(ContentType.Application.Json)
        setBody(CreateChatRequest(type = "group", memberUids = lis-
tOf(bobUid), name = "Squad"))
    }

    assertEquals(HttpStatusCode.Created, response.status)
    val chat: Chat = response.body()
    assertEquals("group", chat.type)
}

// — 9. getChatsReturnsList —————
@Test

```

```

    fun `getChatsReturnsList - 200 со списком чатов пользователя`() =
runTestApp { mocks ->
    val client = jsonClient()
    val chats = listOf(makeChat(name = "Family"), makeChat(name =
"Work"))
    coEvery { mocks.chats.getUserChats(aliceUid) } returns chats

    val response = client.get("/chats") { bearer(fakeToken(aliceUid)) }

    assertEquals(HttpStatusCode.OK, response.status)
    val result: List<Chat> = response.body()
    assertEquals(2, result.size)
}

// — 10. getChatByIdForNonMemberFails403 —————
@Test
fun `getChatByIdForNonMemberFails403 - 403 при доступе к чужому чату`() =
runTestApp { mocks ->
    val client = jsonClient()
    val foreignChatId = UUID.randomUUID()
    coEvery { mocks.chats.isUserInChat(foreignChatId, aliceUid) } returns
false

    val response = client.get("/chats/$foreignChatId") { bear-
er(fakeToken(aliceUid)) }

    assertEquals(HttpStatusCode.Forbidden, response.status)
}

// — 11. sendMessageSuccess —————
@Test
fun `sendMessageSuccess - 201 и рассылка через WebSocket`() = runTestApp
{ mocks ->
    val client = jsonClient()
    val chatId = UUID.randomUUID()
    val msg = makeMessage(chatId, aliceUid, "Hi Bob!")
    coEvery { mocks.chats.isUserInChat(chatId, aliceUid) } returns true
    coEvery { mocks.messages.sendMessage(chatId, aliceUid, any()) } re-
turns msg
    coEvery { mocks.chats.getChatMembers(chatId) } returns listOf(
        ChatMember(aliceUid, "Alice", role = "admin", joinedAt = now),
        ChatMember(bobUid, "Bob", role = "member", joinedAt = now)
    )

    val response = client.post("/chats/$chatId/messages") {
        bearer(fakeToken(aliceUid))
        contentType(ContentType.Application.Json)
        setBody(SendMessageRequest(content = "Hi Bob!"))
    }

    assertEquals(HttpStatusCode.Created, response.status)
    val saved: Message = response.body()
    assertEquals("Hi Bob!", saved.content)
    coVerify(exactly = 1) { mocks.ws.sendToUsers(match {
it.containsAll(listOf(aliceUid, bobUid)) }, any()) }
}

// — 12. sendMessageInForeignChatFails403 —————
@Test
fun `sendMessageInForeignChatFails403 - 403 при отправке в чужой чат`() =
runTestApp { mocks ->
    val client = jsonClient()
    val chatId = UUID.randomUUID()

```

```

        coEvery { mocks.chats.isUserInChat(chatId, aliceUid) } returns false

        val response = client.post("/chats/$chatId/messages") {
            bearer(fakeToken(aliceUid))
            contentType(ContentType.Application.Json)
            setBody(SendMessageRequest(content = "spam"))
        }

        assertEquals(HttpStatusCode.Forbidden, response.status)
        coVerify(exactly = 0) { mocks.messages.sendMessage(any(), any(),
any()) }
    }

    // — 13. getMessagesReturnsList —————
    @Test
    fun `getMessagesReturnsList - 200 с историей сообщений`() = runTestApp {
mocks ->
        val client = jsonClient()
        val chatId = UUID.randomUUID()
        val msgs = listOf(makeMessage(chatId, aliceUid, "1"), makeMes-
sage(chatId, bobUid, "2"))
        coEvery { mocks.chats.isUserInChat(chatId, aliceUid) } returns true
        coEvery { mocks.messages.getChatMessages(chatId, 50, null) } returns
msgs

        val response = client.get("/chats/$chatId/messages") { bear-
er(fakeToken(aliceUid)) }

        assertEquals(HttpStatusCode.OK, response.status)
        val result: List<Message> = response.body()
        assertEquals(2, result.size)
    }

    // — 14. editOwnMessageSuccess —————
    @Test
    fun `editOwnMessageSuccess - 200 при правке собственного сообщения`() =
runTestApp { mocks ->
        val client = jsonClient()
        val chatId = UUID.randomUUID()
        val msgId = UUID.randomUUID()
        val original = makeMessage(chatId, aliceUid, "old", id = msgId)
        val updated = original.copy(content = "new", isEdited = true)
        coEvery { mocks.messages.getMessageById(msgId) } returns original
        coEvery { mocks.messages.editMessage(msgId, "new") } returns updated
        coEvery { mocks.chats.getChatMembers(chatId) } returns emptyList()

        val response = client.put("/chats/$chatId/messages/$msgId") {
            bearer(fakeToken(aliceUid))
            contentType(ContentType.Application.Json)
            setBody(EditMessageRequest(content = "new"))
        }

        assertEquals(HttpStatusCode.OK, response.status)
        val msg: Message = response.body()
        assertEquals("new", msg.content)
        assertTrue(msg.isEdited)
    }

    // — 15. editForeignMessageFails403 —————
    @Test
    fun `editForeignMessageFails403 - 403 при правке чужого сообщения`() =
runTestApp { mocks ->
        val client = jsonClient()

```

```

        val chatId = UUID.randomUUID()
        val msgId = UUID.randomUUID()
        val foreign = makeMessage(chatId, bobUid, "bob's msg", id = msgId)
        coEvery { mocks.messages.getMessageById(msgId) } returns foreign

        val response = client.put("/chats/$chatId/messages/$msgId") {
            bearer(fakeToken(aliceUid))
            contentType(ContentType.Application.Json)
            setBody(EditMessageRequest(content = "hijacked"))
        }

        assertEquals(HttpStatusCode.Forbidden, response.status)
        coVerify(exactly = 0) { mocks.messages.editMessage(any(), any()) }
    }

    // — 16. deleteOwnMessageSuccess —————
    @Test
    fun `deleteOwnMessageSuccess - 204 при удалении своего сообщения`() =
        runTestApp { mocks ->
            val client = jsonClient()
            val chatId = UUID.randomUUID()
            val msgId = UUID.randomUUID()
            val msg = makeMessage(chatId, aliceUid, id = msgId)
            coEvery { mocks.messages.getMessageById(msgId) } returns msg
            coEvery { mocks.messages.deleteMessage(msgId) } returns Unit
            coEvery { mocks.chats.getChatMembers(chatId) } returns emptyList()

            val response = client.delete("/chats/$chatId/messages/$msgId") {
                bearer(fakeToken(aliceUid))
            }

            assertEquals(HttpStatusCode.NoContent, response.status)
            coVerify(exactly = 1) { mocks.messages.deleteMessage(msgId) }
        }

    // — 17. unauthorisedAccessFails401 —————
    @Test
    fun `unauthorisedAccessFails401 - 401 без токена авторизации`() =
        runTestApp { _ ->
            val client = jsonClient()
            val r1 = client.get("/auth/me")
            val r2 = client.get("/chats")
            val r3 = client.post("/chats") {
                contentType(ContentType.Application.Json)
                setBody(CreateChatRequest(type = "group", memberUids = lis-
toOf(bobUid), name = "X"))
            }
            val r4 = client.post("/chats/${UUID.randomUUID()}/messages") {
                contentType(ContentType.Application.Json)
                setBody(SendMessageRequest(content = "hi"))
            }

            assertEquals(HttpStatusCode.Unauthorized, r1.status)
            assertEquals(HttpStatusCode.Unauthorized, r2.status)
            assertEquals(HttpStatusCode.Unauthorized, r3.status)
            assertEquals(HttpStatusCode.Unauthorized, r4.status)
        }

    // — 18. searchUsersByQuerySuccess —————
    @Test
    fun `searchUsersByQuerySuccess - 200 с результатами поиска`() =
        runTestApp { mocks ->
            val client = jsonClient()

```

```

        coEvery { mocks.users.searchUsers("ali", 20) } returns listOf(
            makeUser(aliceUid, "alice"),
            makeUser("uid2", "alicia")
        )

        val response = client.get("/users/search?q=ali") { bearer(fakeToken(aliceUid)) }

        assertEquals(HttpStatusCode.OK, response.status)
        val users: List<User> = response.body()
        assertEquals(2, users.size)
        assertTrue(users.all { it.username.startsWith("ali") })
    }

    // — 19. searchUsersWithoutQueryFails400 —————
    @Test
    fun `searchUsersWithoutQueryFails400 - 400 если не передан параметр q`() =
        runTestApp { _ ->
            val client = jsonClient()
            val response = client.get("/users/search") { bearer(fakeToken(aliceUid)) }
            assertEquals(HttpStatusCode.BadRequest, response.status)
        }

    // — 20. unknownRouteReturns404 —————
    @Test
    fun `unknownRouteReturns404 - 404 для несуществующего маршрута`() =
        runTestApp { _ ->
            val client = jsonClient()
            val response = client.get("/nonexistent")
            assertEquals(HttpStatusCode.NotFound, response.status)
        }
    }
}

/**
 * Обёртка над testApplication, которая:
 * - перебивает environment на пустой MapApplicationConfig (иначе Ktor сам
    подхватит
 * - основной module() из application.conf и попыбует подключиться к PostgreSQL);
 * - регистрирует testModule(mocks);
 * - даёт блоку доступ к mocks для настройки coEvery {...}.
 */
fun runTestApp(block: suspend ApplicationTestBuilder.(TestMocks) -> Unit) =
    testApplication {
        environment { config = MapApplicationConfig() }
        val mocks = TestMocks()
        application { testModule(mocks) }
        block(mocks)
    }
}

```